

Apuntes clase 2

Sistemas Operativos

Universidad de Antioquia

Facultad de Ingeniería

Ingeniería de Sistemas

Agenda

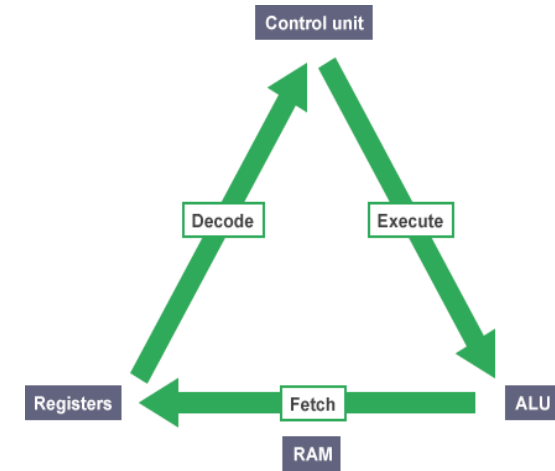
- ¿Qué hace un Sistema Operativo?
- Definición de los Sistemas Operativos
- Virtualización de Recursos
- Operaciones del sistema operativo
- Objetivos de diseño de los Sistemas Operativos

¿Que hace un sistema operativo?

¿Como se ejecuta un programa?

Para esto la CPU lleva cabo los siguientes pasos:

1. **Fetch:** Obtiene la instrucción de memoria.
2. **Decode:** Averigua la instrucción a ejecutar.
3. **Exec:** Realiza la operación indicada.
4. Se salta a la próxima instrucción y se repite el procedimiento



Ver video: The Fetch-Execute Cycle: What's Your Computer Actually Doing? ([link](#))

¿Que hace un sistema operativo?

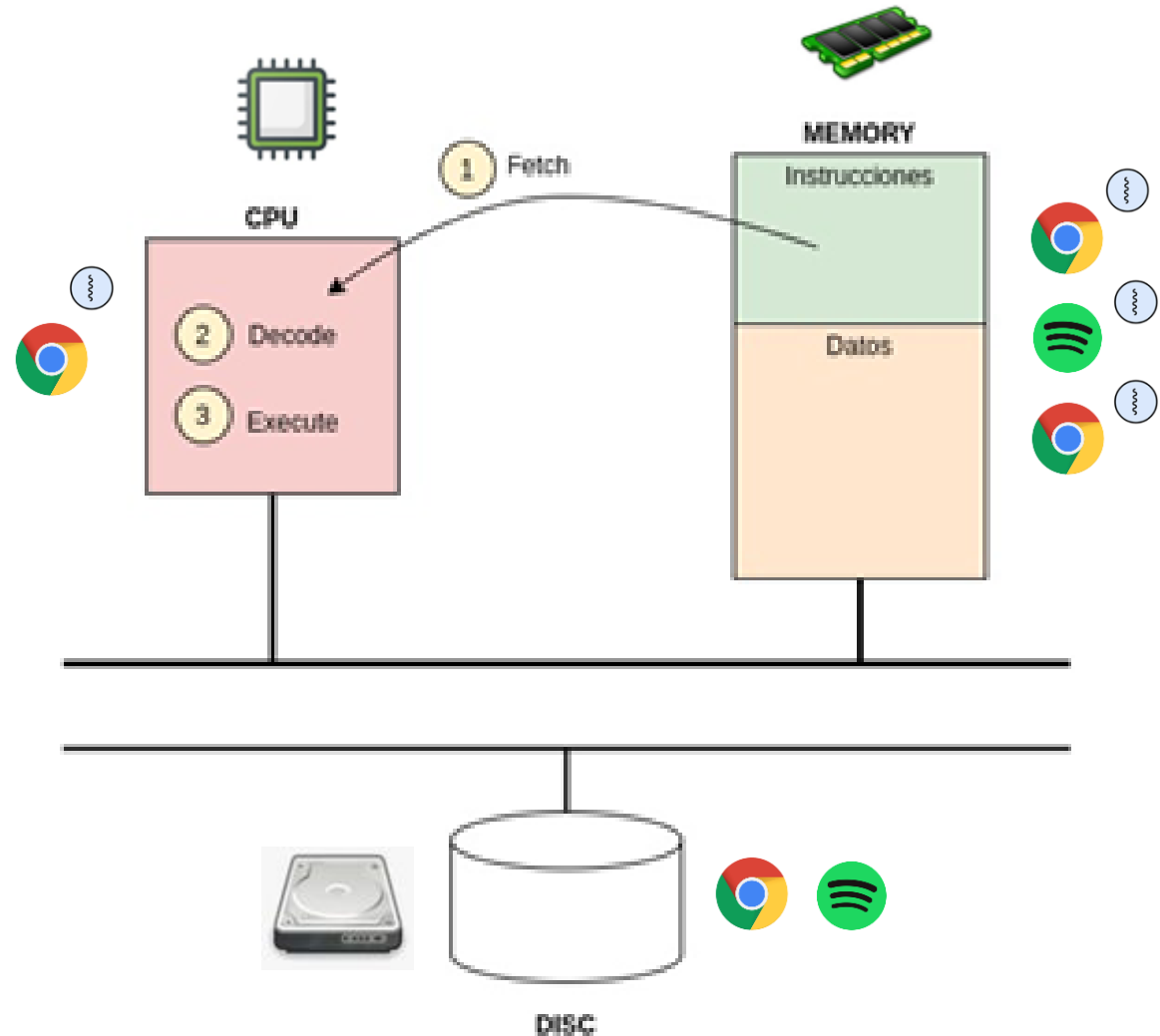
¿Como se ejecuta un programa?

MIPS

```
.data
    x: .word 1
    y: .word 0

.text
main:
    lw    $t0 x
    li    $t1 2
    add   $t1 $t0 $t1
    sw    $t1, y
    jr    $ra
```

<https://wepsim.github.io/>

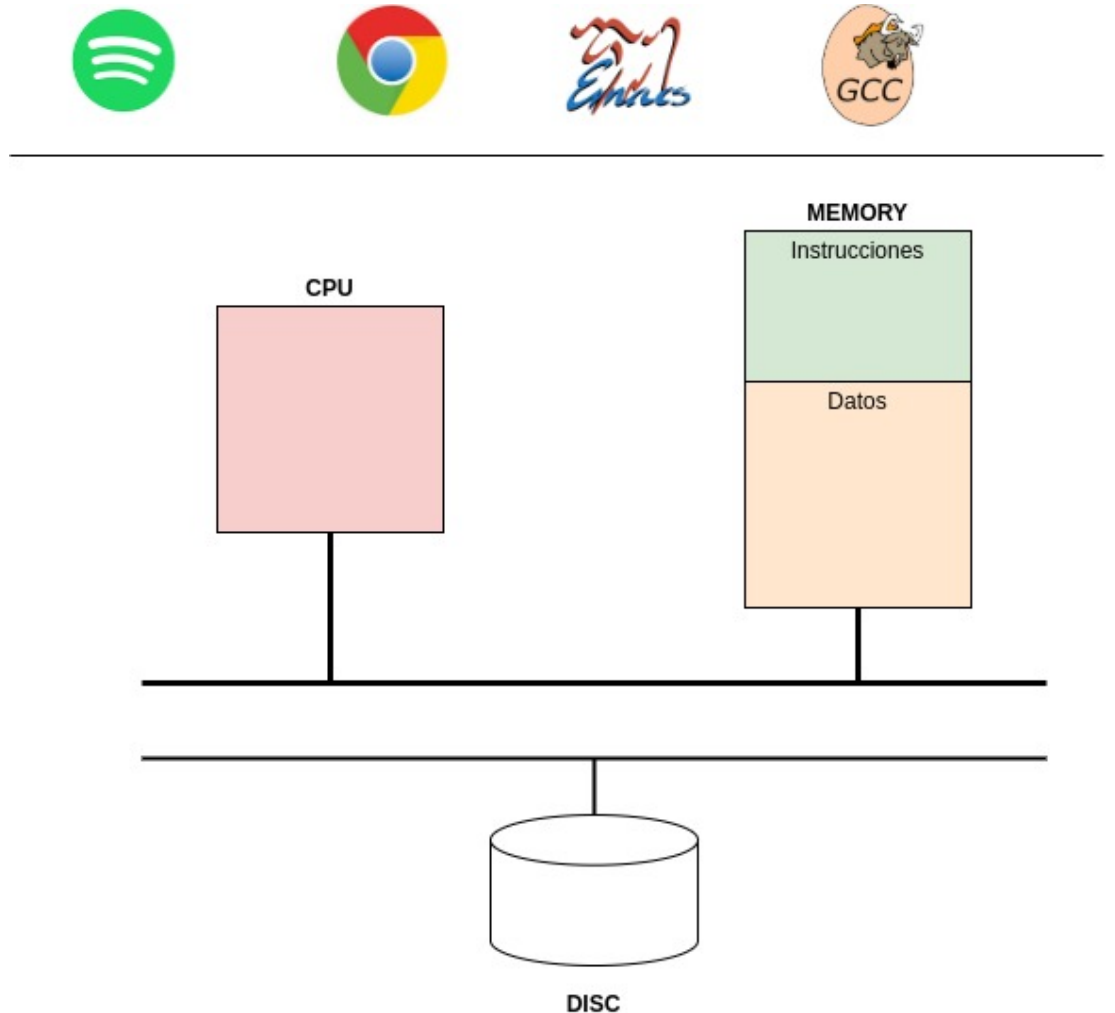


¿Que hace un sistema operativo?

¿Como facilitar la ejecución de programas en un computador?



Incluso, ¿cómo permitir que se puedan ejecutar **varios programas** al mismo tiempo?



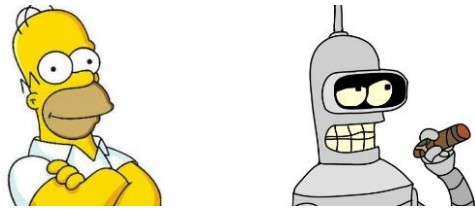
¿Que hace un sistema operativo?

Estructura de los sistemas de cómputo

Componentes principales:

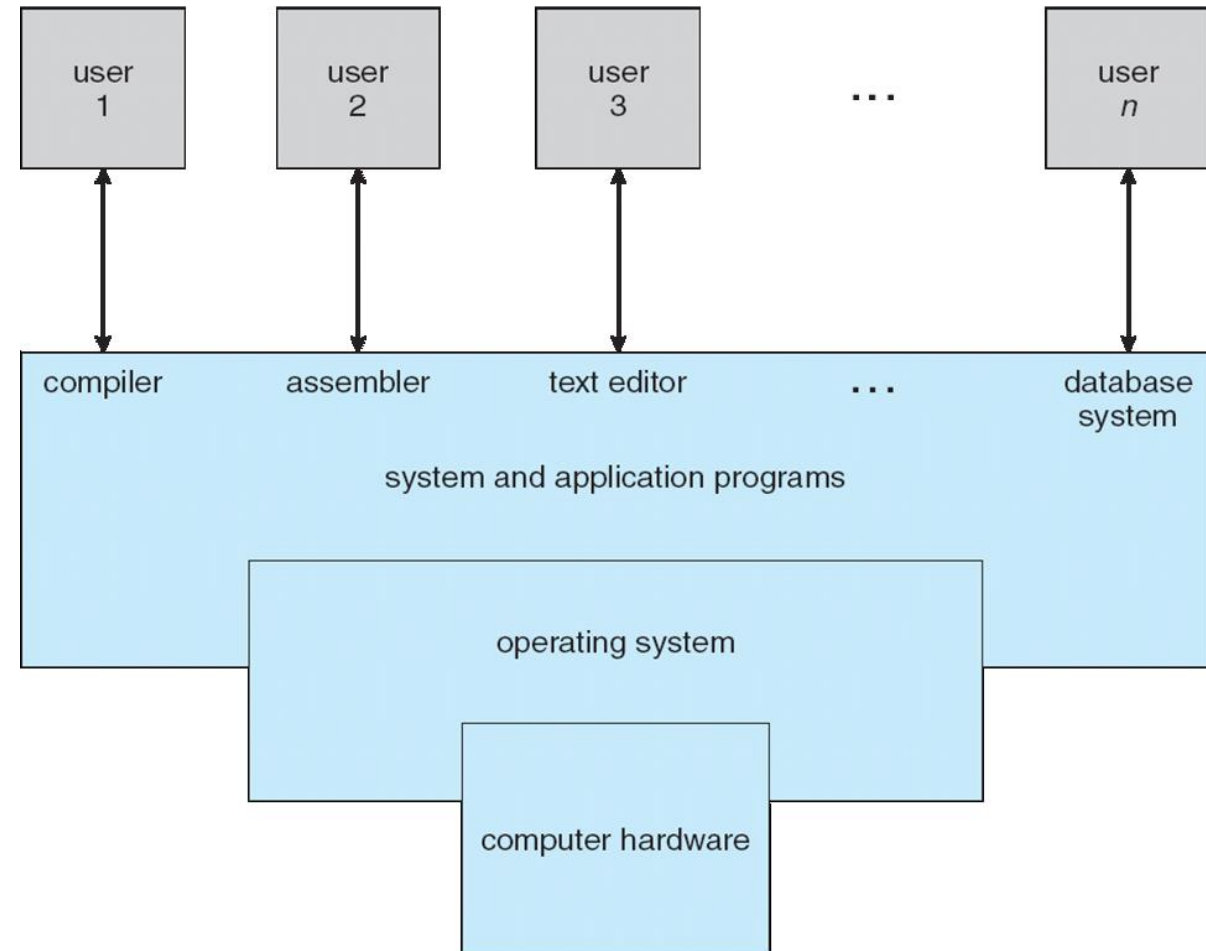
1. Usuarios

Personas, máquinas, otras computadoras



2. Aplicaciones

Compiladores, procesadores de texto, navegadores, etc.



¿Que hace un sistema operativo?

Estructura de los sistemas de cómputo

Componentes principales:

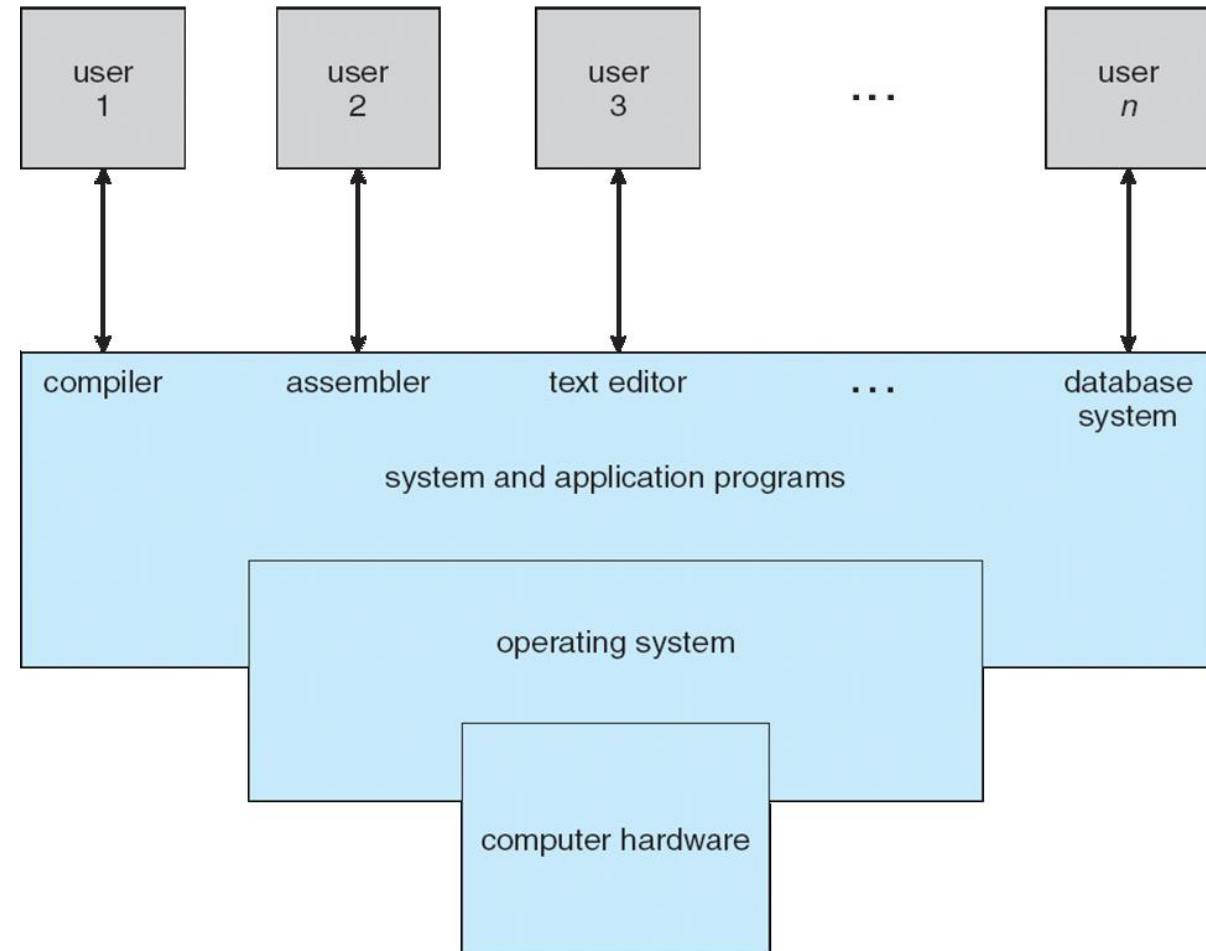
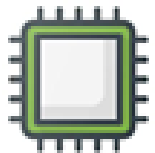
3. Sistema Operativo

Controla y coordina el uso del hardware entre varias aplicaciones y usuarios



3. Hardware

CPU - Memoria - Dispositivos E/S



¿Que hace un sistema operativo?

Clasificación de los sistemas de cómputo

- Depende del punto de vista

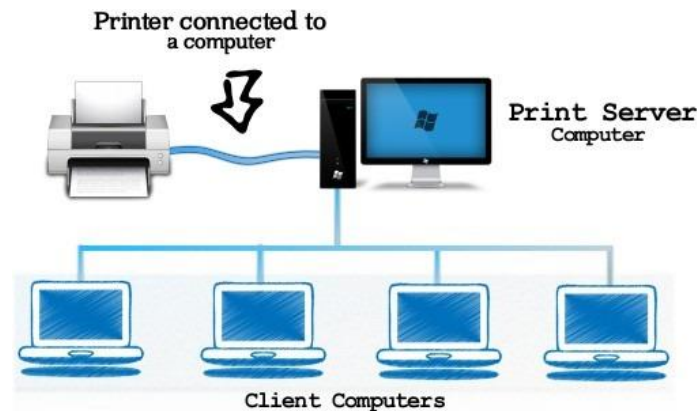


Usuarios

No importa mucho la utilización de los recursos

Computadora compartida

Mantener a todos los usuarios felices



Sistema dedicado

Compromiso entre usabilidad y utilización de recursos



¿Que hace un sistema operativo?

Clasificación de los sistemas de cómputo

- Depende del punto de vista

Dispositivo portátil

Gestionar recursos limitados,
optimizar la usabilidad



Sistemas embebidos

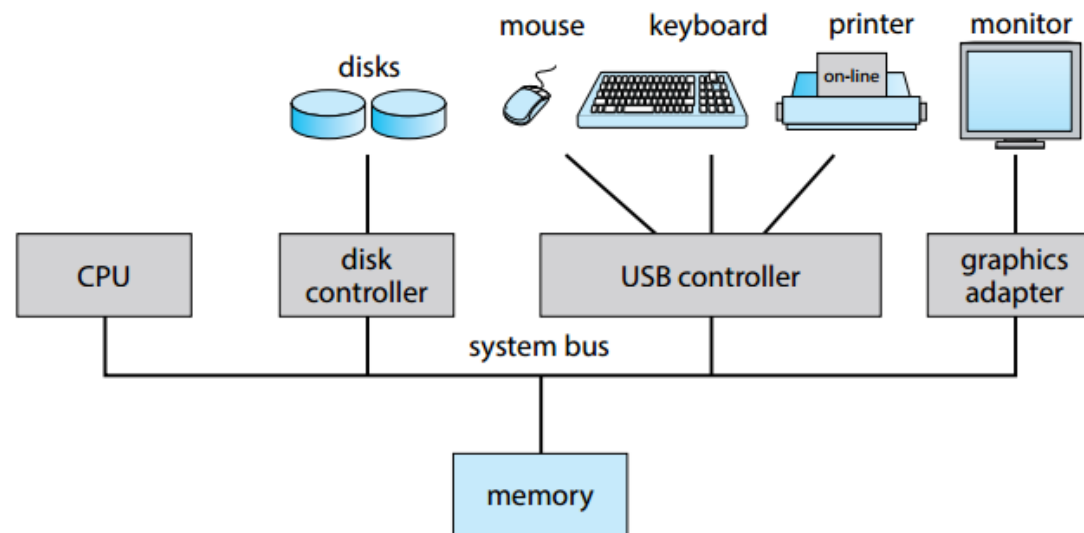
Interfaz de usuario muy limitada o
inexistente



Definición de Sistema Operativo

Es un **software** que:

1. Administrar eficiente y justa (fair) **recursos**.



1. Se encarga de controlar la ejecución de programas
 - **Previene** errores.
 - Evita el **uso inadecuado** del sistema de computo

Definición de Sistema Operativo

No hay una definición universalmente aceptada:



Tanenbaum:

*An OS is a software that “performs two essentially unrelated functions: providing application programmers a **clean abstract set** of resources instead of the messy hardware ones and **managing** these hardware resources”*



Arpaci-Dusseau:

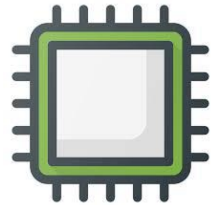
*An OS is a body of software that “is responsible for making it easy to run programs, allowing programs to share memory, enabling programs to interact with devices, ..., as it is in charge of making sure the system operates **correctly** and **efficiently** in an **easy-to-use manner**”*

Virtualización de recursos

Sistema operativo como máquina virtual

- El sistema operativo toma un **recurso físico** y lo transforma en una **forma virtual** de sí mismo.

Recurso físico



Procesador



Memoria



Disco

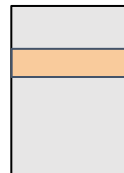
...

- La forma virtual es más **general**, **potente** y **fácil de usar**.

Forma Virtual



Proceso



Espacio de
direccionamient



Archivo

...

Virtualización de recursos

Llamadas al sistema

- Las **llamadas al sistema** permiten que el usuario pueda **decirle** al Sistema Operativo **qué hacer**.
- El sistema operativo proporciona una interfaz (**API**, librería estándar).
- Un sistema operativo típico proporciona cientos de llamadas al sistema:
 - Ejecutar programas
 - Acceder a memoria
 - Acceder a los dispositivos
- Ejemplo: [POSIX API](#)

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS		
The following illustrates various equivalent system calls for Windows and UNIX operating systems.		
	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communications	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Virtualización de recursos

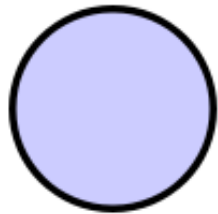
Administrador de recursos

- El SO administra los **recursos de un sistema** de cómputo: procesador, memoria, disco duro.
- El SO permite:
 - La ejecución de múltiples programas → **compartiendo la CPU**
 - Que múltiples programas accedan simultáneamente a sus datos e instrucciones → **compartiendo la memoria**
 - Que múltiples programas accedan a los dispositivos → **compartiendo los discos**

Virtualización de recursos

Virtualización de la CPU

- El sistema tiene un gran número de CPUs virtuales.
 - Convierte una CPU en “**infinitas**” CPUs virtuales
 - Permite que múltiples programas se ejecuten “**de manera concurrente**”
- Convención



Proceso



Hilo

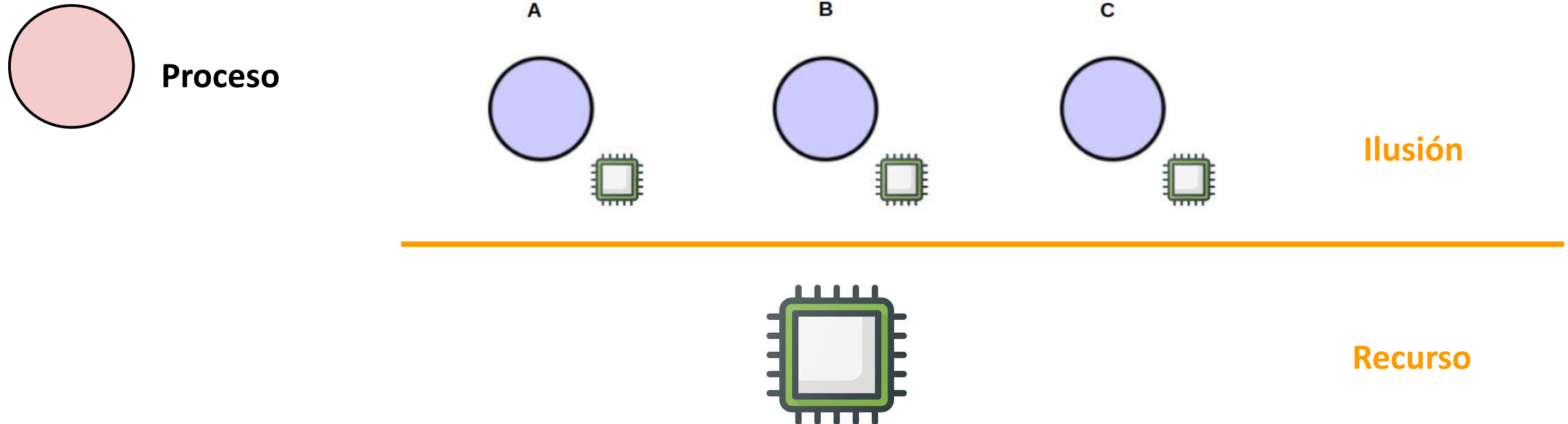


Archivo
ejecutable

Virtualización de recursos

Virtualización de la CPU

- El sistema tiene un gran número de CPUs virtuales.
 - Convierte una CPU en “**infinitas**” CPUs virtuales
 - Permite que múltiples programas se ejecuten “**de manera concurrente**”



Virtualización de recursos

Ejemplo



cpu.c

```
#include <stdio.h>
#include <stdlib.h>
#include "common.h"

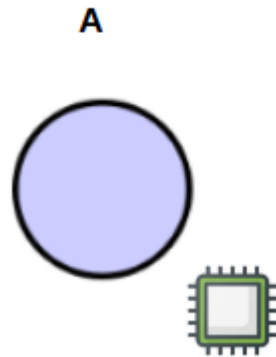
int main(int argc, char *argv[])
{
    if (argc != 2) {
        fprintf(stderr, "usage: cpu <string>\n");
        exit(1);
    }
    char *str = argv[1];

    while (1) {
        printf("%s\n", str);
        Spin(1);
    }
    return 0;
}
```

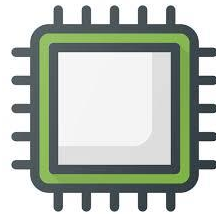
Virtualización de recursos

Virtualización de la CPU

Ilusión



Recurso



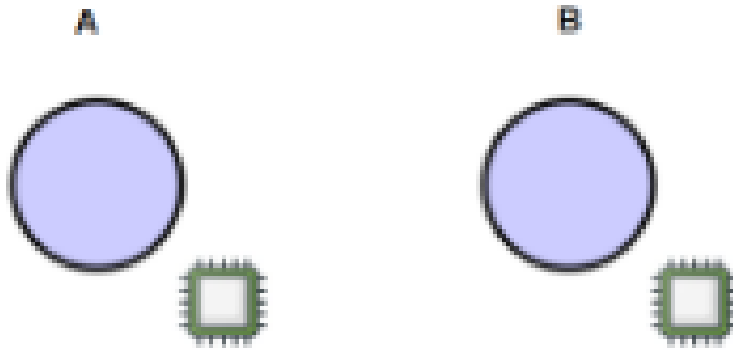
```
examples -- a.out hola -- 80x20
Dannys-MacBook-Pro:examples dannymunera$ ./a.out
usage: cpu <string>
Dannys-MacBook-Pro:examples dannymunera$ emacs 1-cpu.c
Dannys-MacBook-Pro:examples dannymunera$ ./a.out hola
hola
hola
hola
hola
hola
hola
hola
hola
```

Ver video: C01P10 - Virtualización de la CPU: Ejemplos - ISI485 Sistemas Operativos ([link](#))

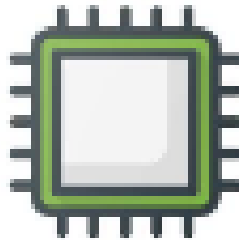
Virtualización de recursos

Virtualización de la CPU

Ilusión



Recurso

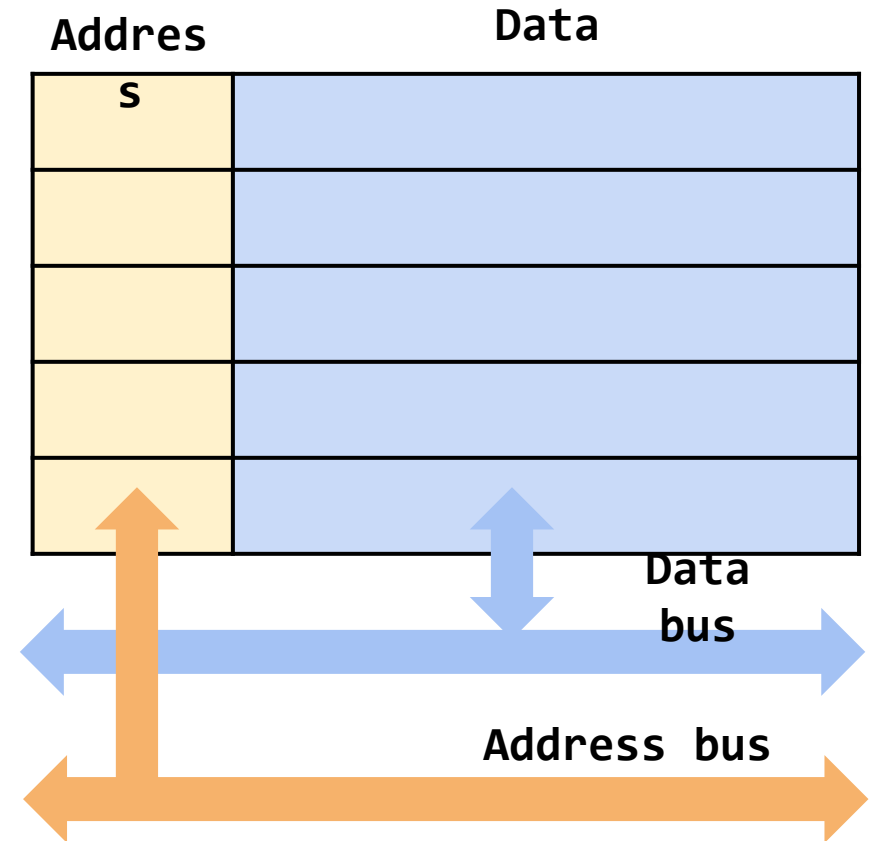


```
examples — bash — 80x26
Dannys-MacBook-Pro:examples dannymunera$ ./a.out hola & ./a.out mundo
[1] 1301
mundo
hola
mundo
hola
mundo
hola
mundo
hola
mundo
hola
mundo
hola
mundo
hola
mundo
hola
^C
```

Virtualización de recursos

Virtualización de Memoria

- La memoria física **es un arreglo de bytes**.
- Un programa almacena todas sus estructuras de datos en la memoria:
 - Lee la memoria (**load**): Especifica una dirección para poder acceder a un dato
 - Escribe en la memoria (**write**): Especifica el dato para ser escrito en una dirección dada.



Virtualización de recursos

Ejemplo



mem.c

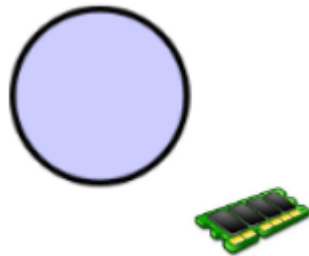
```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>
#include "common.h"

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: mem <value>\n");
        exit(1);
    }
    int *p;
    p = malloc(sizeof(int));
    assert(p != NULL);
    printf("(%d) addr pointed to by p: %p\n", (int) getpid(), p);
    *p = atoi(argv[1]); // assign value to addr stored in p
    while (1) {
        Spin(1);
        *p = *p + 1;
        printf("(%d) value of p: %d\n", getpid(), *p);
    }
    return 0;
}
```

Virtualización de recursos

Virtualización de Memoria

Ilusión



```
Dannys-MacBook-Pro:examples dannymunera$ gcc -o 2-mem 2-mem.c
Dannys-MacBook-Pro:examples dannymunera$ ./2-mem
(1335) address pointed to by p: 0x100400190
(1335) p: 0x100400190
(1335) p: 0x100400190
(1335) p: 0x100400190
(1335) p: 0x100400190
(1335) p: 0x100400190
(1335) p: 0x100400190
```

Recurso



Ver video: C01P11 - Ejemplo Virtualización de Memoria - ISI485 Sistemas Operativos ([link](#))

Virtualización de recursos

Virtualización de Memoria

Ilusión



```
Dannys-MacBook-Pro:examples dannymunera$ ./2-men & ./2-men
[1] 1336
(1336) address pointed to by p: 0x100100000
(1337) address pointed to by p: 0x100100000
(1336) p: 0x100100000
(1337) p: 0x100100000
(1337) p: 0x100100000
(1336) p: 0x100100000
(1336) p: 0x100100000
(1337) p: 0x100100000
(1337) p: 0x100100000
```

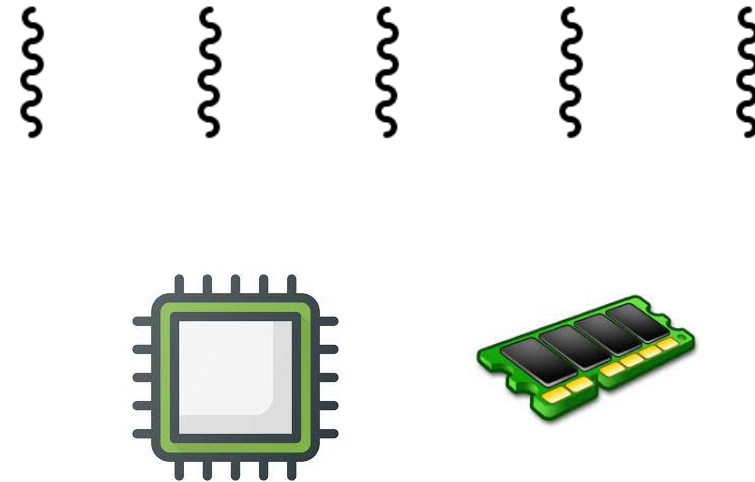
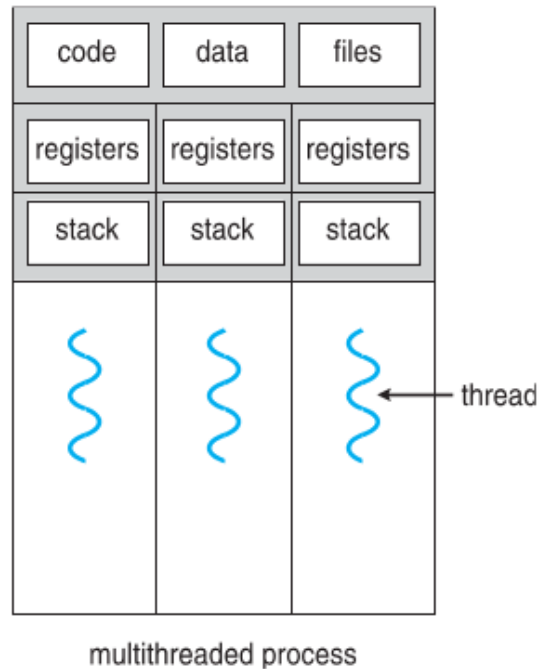
Recurso



Virtualización de recursos

Recurrencia

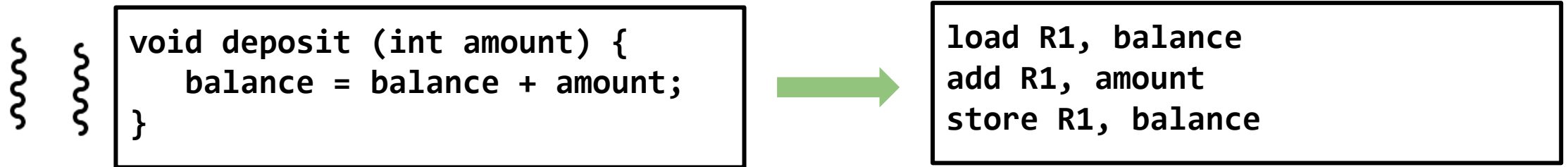
- El SO está realizando muchas cosas a la vez, primero ejecutando un proceso, luego otro, y así sucesivamente.
- Programas modernos multi-hilo presentan **problemas de concurrencia**.



Virtualización de recursos

Recurrencia

- **Contextualización:** Dos hilos (**threads**) comparten el balance de una cuenta en memoria). Cada hilo corre su propio código llamado **deposit()** tal y como se muestra a continuación.



Responda:

- ¿Cuales son variables compartidas?
- ¿Cuales con variables privadas?

Virtualización de recursos

Recurrencia

- Incrementar una **variable compartida** usa tres instrucciones:
 - **Cargar** el valor del contador de la memoria a un registro
 - **Incrementar**
 - **Almacenar** el valor modificado en la memoria

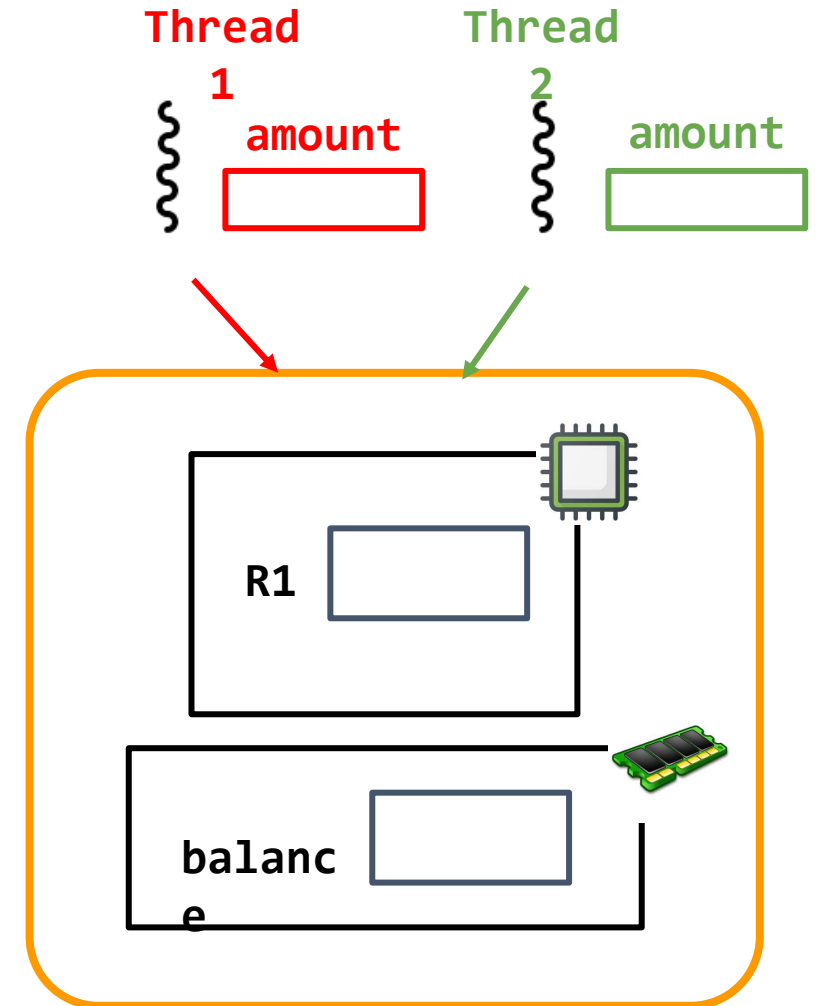
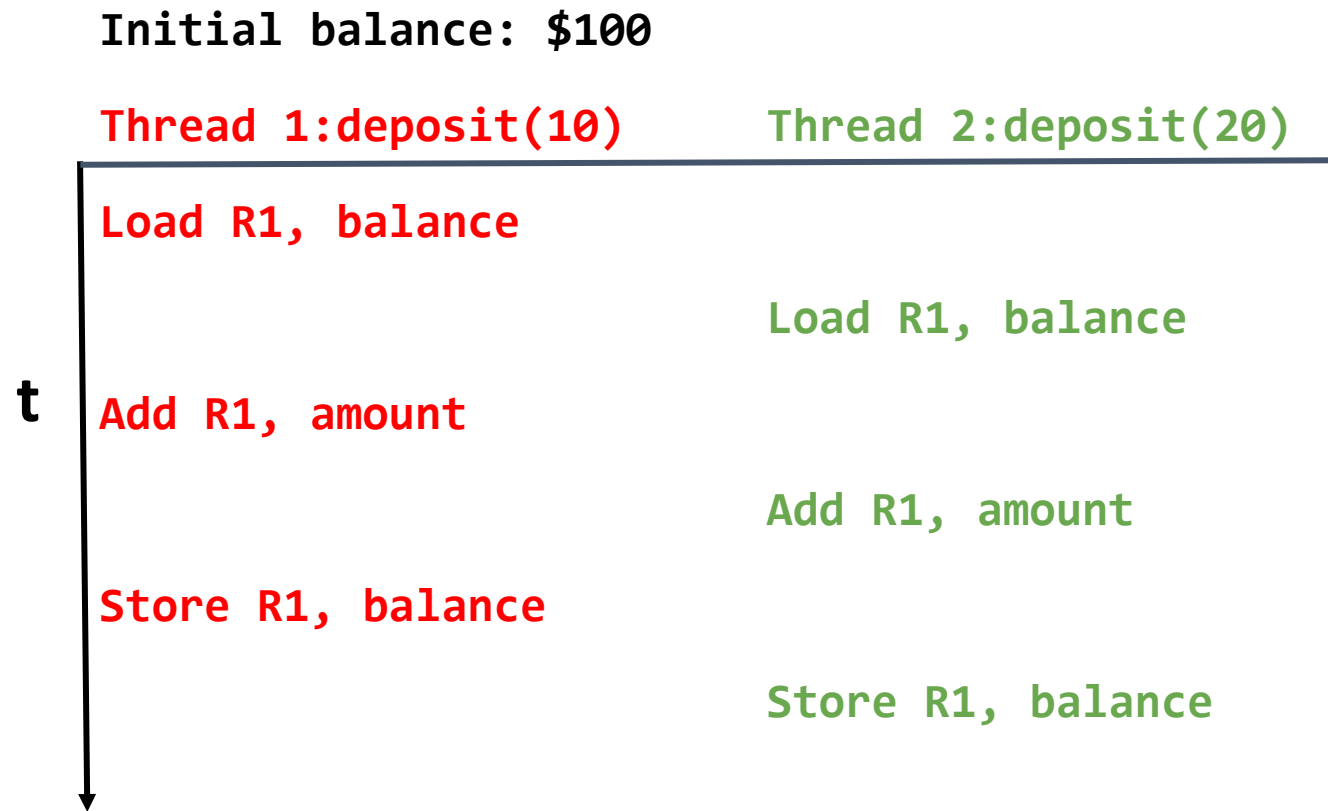
```
void deposit (int amount) {  
    balance = balance + amount;  
}
```



```
load R1, balance  
add R1, amount  
store R1, balance
```

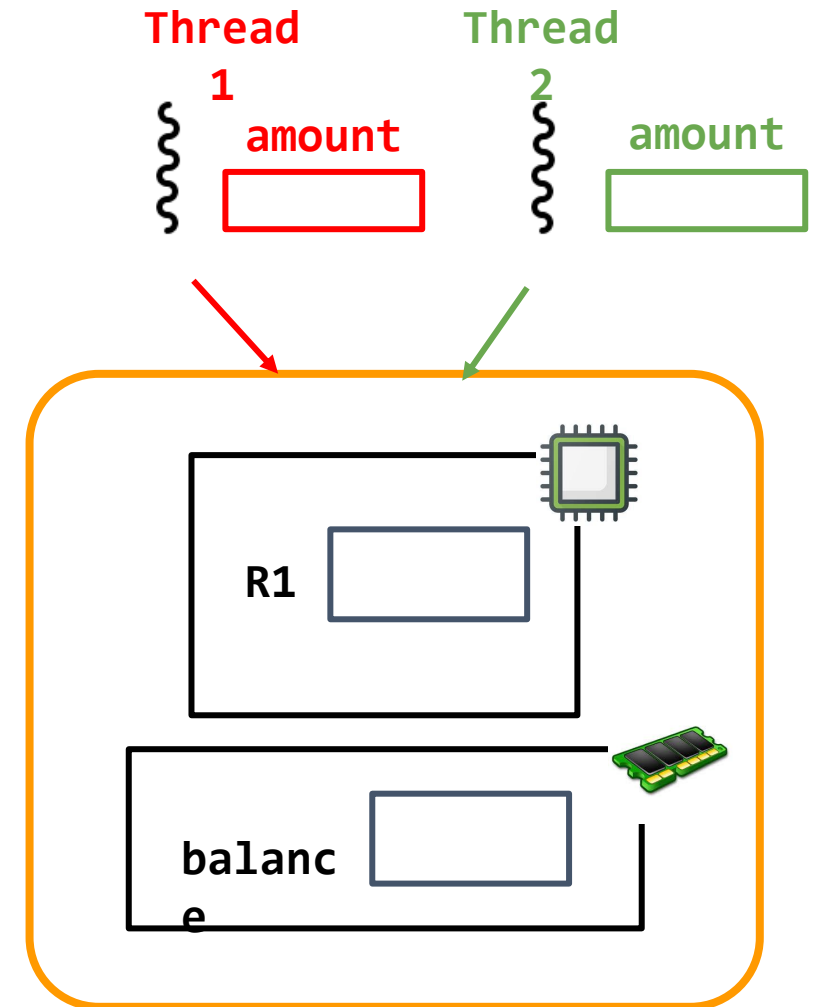
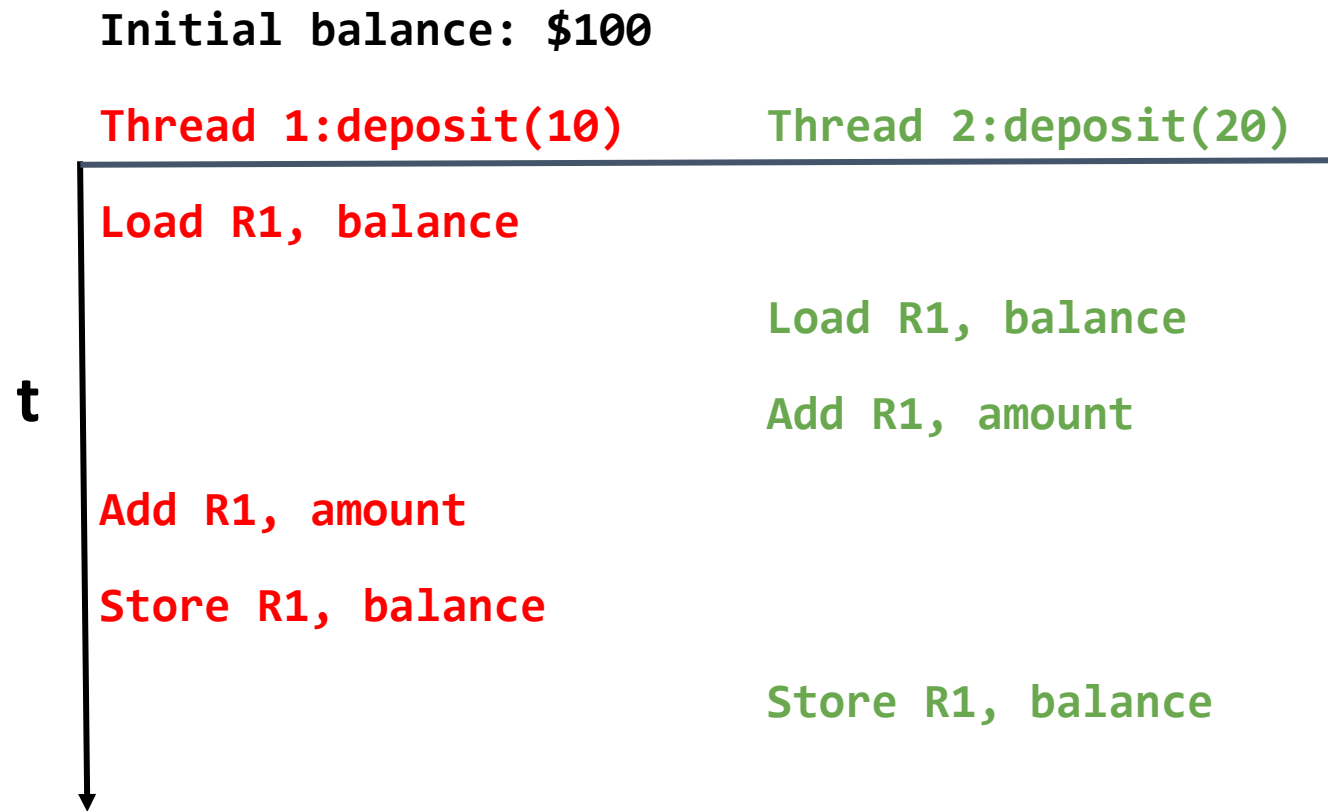
Virtualización de recursos

Recurrencia



Virtualización de recursos

Recurrencia



Virtualización de recursos

Ejemplo



threads.c

```
#include <stdio.h>
#include <stdlib.h>
#include "common.h"
#include "common_threads.h"
```

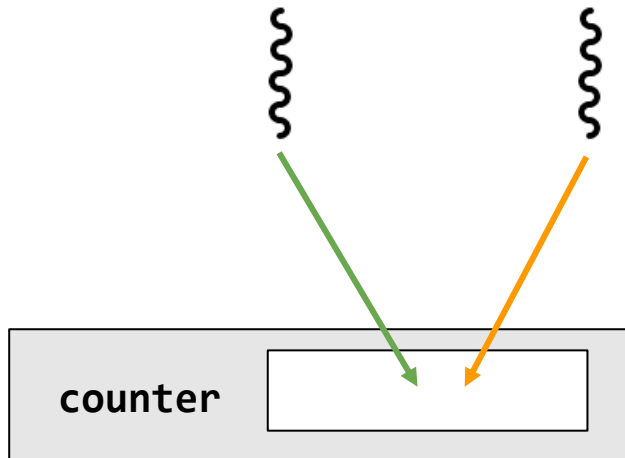
```
volatile int counter = 0;
int loops;
```

```
void *worker(void *arg) {
    int i;
    for (i = 0; i < loops;
i++) {
        counter++;
    }
    return NULL;
}
```

```
int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "usage: threads
<loops>\n");
        exit(1);
    }
    loops = atoi(argv[1]);
    pthread_t p1, p2;
    printf("Initial value : %d\n", counter);
    Pthread_create(&p1, NULL, worker, NULL);
    Pthread_create(&p2, NULL, worker, NULL);
    Pthread_join(p1, NULL);
    Pthread_join(p2, NULL);
    printf("Final value   : %d\n", counter);
    return 0;
}
```

Virtualización de recursos

Ejemplo concurrencia



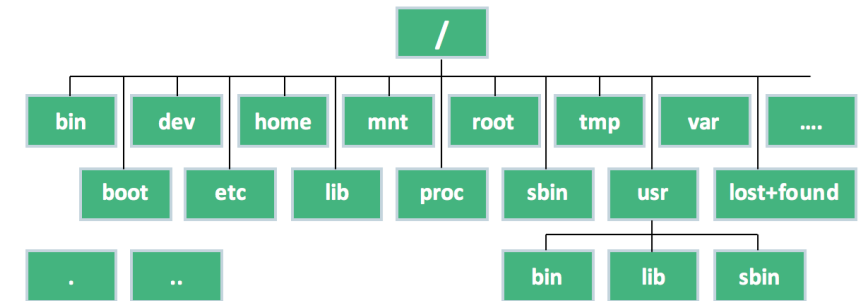
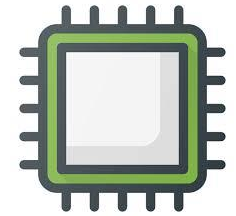
```
Dannys-MacBook-Pro:examples dannymunera$ ./threads 10
Initial value : 0
Final value : 20
Dannys-MacBook-Pro:examples dannymunera$ ./threads 100
Initial value : 0
Final value : 200
Dannys-MacBook-Pro:examples dannymunera$ ./threads 1000
Initial value : 0
Final value : 2000
Dannys-MacBook-Pro:examples dannymunera$ ./threads 10000
Initial value : 0
Final value : 20000
Dannys-MacBook-Pro:examples dannymunera$ ./threads 100000
Initial value : 0
Final value : 156427
Dannys-MacBook-Pro:examples dannymunera$ ./threads 100000
Initial value : 0
Final value : 163340
Dannys-MacBook-Pro:examples dannymunera$ ./threads 100000
Initial value : 0
Final value : 200000
Dannys-MacBook-Pro:examples dannymunera$ ./threads 100000
Initial value : 0
Final value : 139245
Dannys-MacBook-Pro:examples dannymunera$
```

Ver video: C01P11 - C01P12 - Ejemplo concurrencia - ISI485 Sistemas Operativos ([link](#))

Virtualización de recursos

Persistencia

- La memoria principal usa una tecnología de **memoria volátil**.
 - Los datos se pierden si deja de haber alimentación.
- Es necesario tener alguna forma de mantener los datos (persistencia).
 - **Hardware:** permite el almacenamiento de datos (discos, duros, discos de estado sólido, etc).
 - **Software:** **sistema de archivos** (controla el disco). Este software es el responsable de almacenar los archivos que crea el usuario de una manera confiable y eficiente en el disco.



Virtualización de recursos

Ejemplo



io.c

```
#include <stdio.h>
#include <unistd.h>
#include <assert.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <string.h>

int main(int argc, char *argv[]) {
    int fd = open("/tmp/file", O_WRONLY | O_CREAT | O_TRUNC, S_IRUSR | S_IWUSR);
    assert(fd >= 0);
    char buffer[20];
    sprintf(buffer, "hello world\n");
    int rc = write(fd, buffer, strlen(buffer));
    assert(rc == (strlen(buffer)));
    fsync(fd);
    close(fd);
    return 0;
}
```


Virtualización de recursos

Ejemplo persistencia



```
Dannys-MacBook-Pro:examples dannymunera$ ls
1-cpu.c 2-mem 2-mem.c 3-con.c 4-per.c a.out per threads
Dannys-MacBook-Pro:examples dannymunera$
Dannys-MacBook-Pro:examples dannymunera$ ./per
Dannys-MacBook-Pro:examples dannymunera$ ls
1-cpu.c      2-mem.c      4-per.c      file_tmp      threads
2-mem        3-con.c      a.out        per
Dannys-MacBook-Pro:examples dannymunera$ cat file_tmp
hello world
Dannys-MacBook-Pro:examples dannymunera$
```

Ver video: C01P13 - Ejemplo Persistencia - ISI485 Sistemas Operativos ([link](#))

Objetivos de diseño de los SO

1. Construir **abstracciones**:

- Hacen que el sistema sea **fácil de usar!**

2. Proporcionar un **buen desempeño**:

- Minimizar el **sobrecosto** (*overhead*) del SO.
- SO debe proporcionar la virtualización sin generar un sobrecosto excesivo.

3. **Protección** entre aplicaciones:

- **Aislamiento**: Un mal comportamiento de un proceso **no debe afectar otros** (o al sistema)

4. Confiabilidad:

- El SO debería ejecutarse **sin problema** por un plazo largo de tiempo.

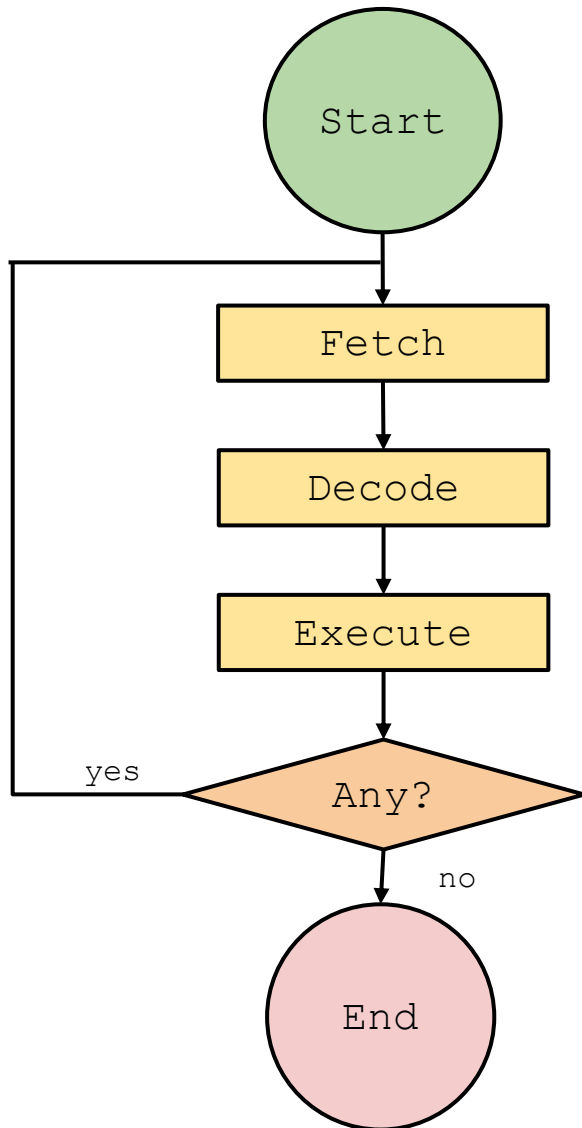
5. Otros:

- Eficiencia energética
- Seguridad
- Movilidad

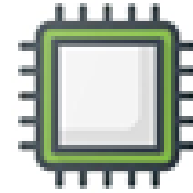
Referencias

- **Operating Systems Three Easy Pieces** ([website](#): Capítulos [1](#) y [2](#))
- Videos de youtube: Clase 1 - Introducción a los sistemas operativos ([link](#)).
- Página de Remzi H. Arpaci-Dusseau ([link](#))
- Operating System Concepts - Tenth Edition ([website](#))
- Operating Systems Notes ([link](#))

Instruction fetch

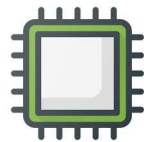
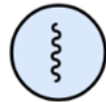
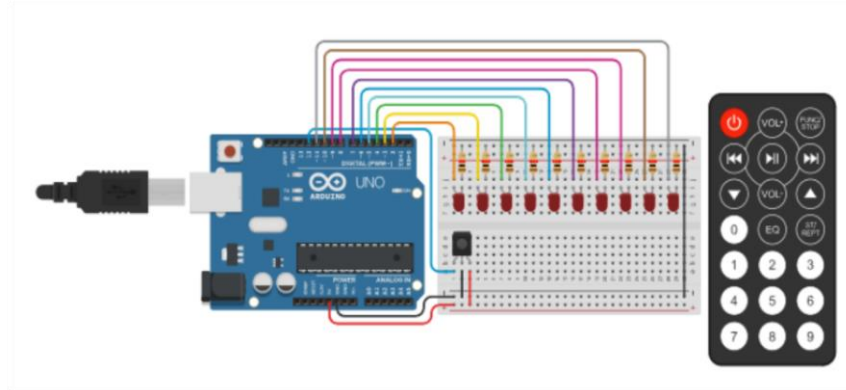


```
.data
    var_x: .word 30
.text
main:
    la $t1, var_x
    lw $t0, 0($t1)
    addi $t0, $t0, 1
    sw $t0, 0($t1)
```

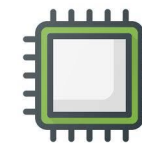
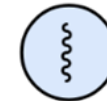
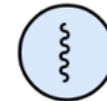
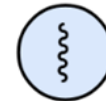


Contraste

Sin sistema Operativo

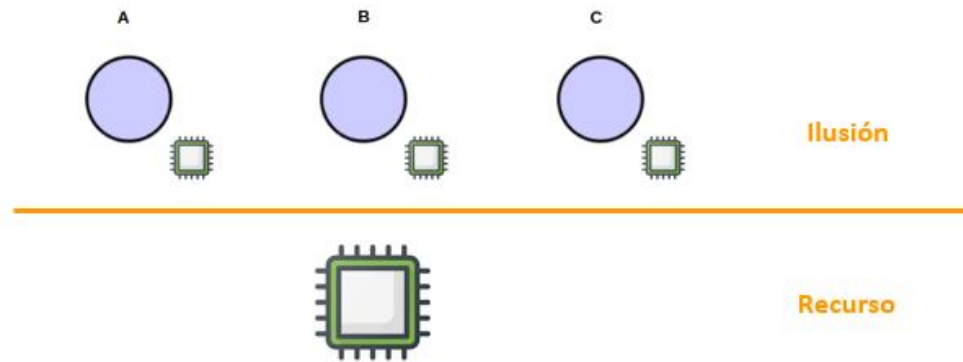


Con sistema Operativo

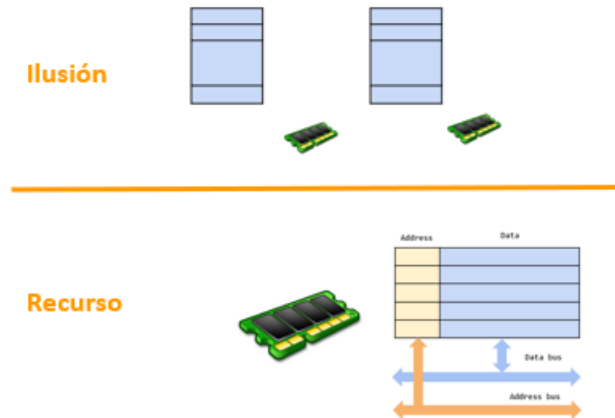


Virtualización

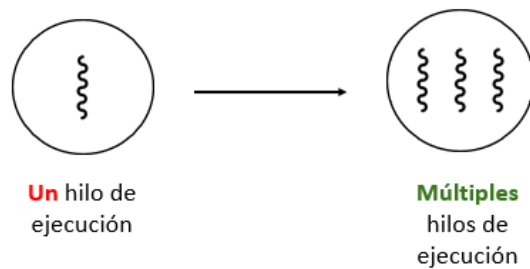
Virtualización de CPU



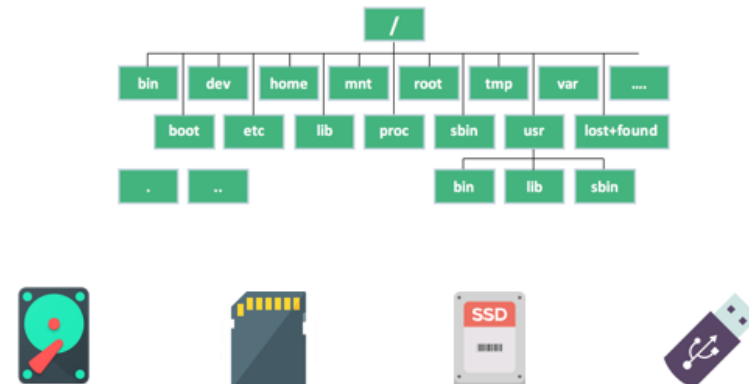
Virtualización de Memoria



Concurrencia y paralelismo



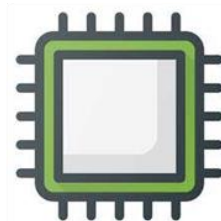
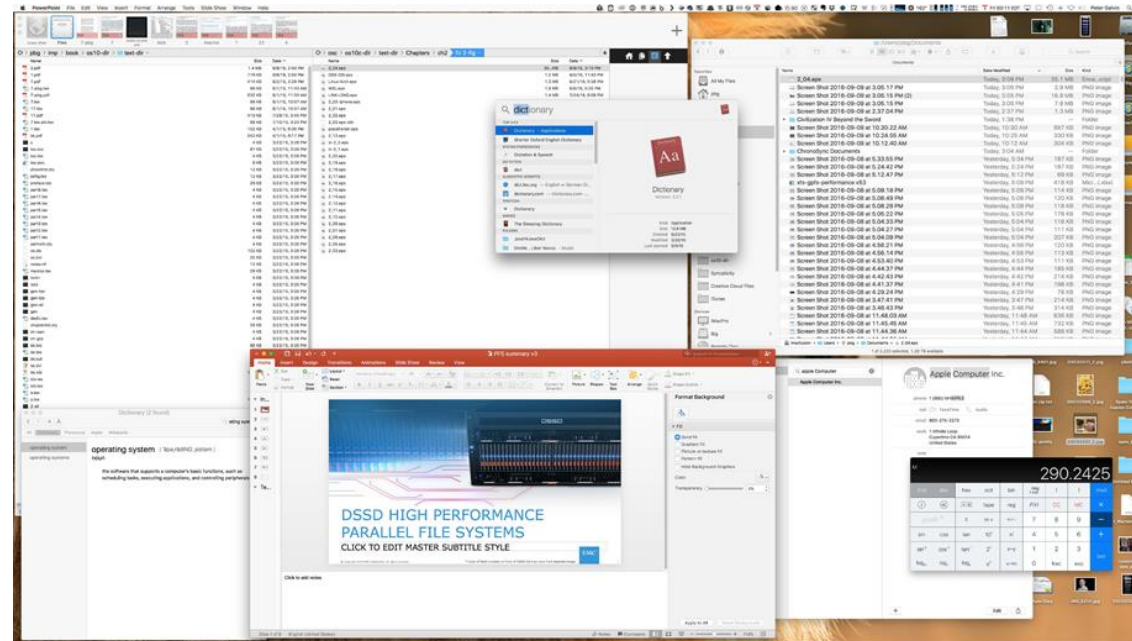
Persistencia



Contextualización

Virtualización de CPU

¿Cómo es que teniendo una sola CPU (en teoría) puedo correr varias tareas a la vez?

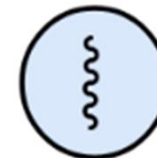
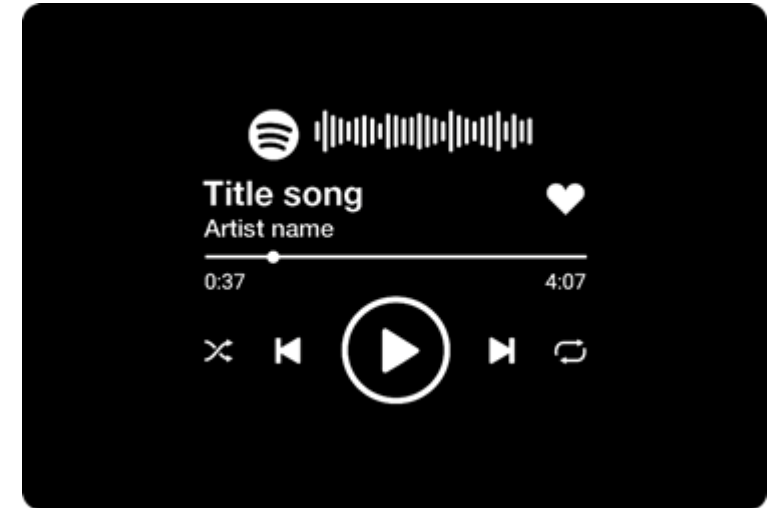


Programa .vs. Proceso

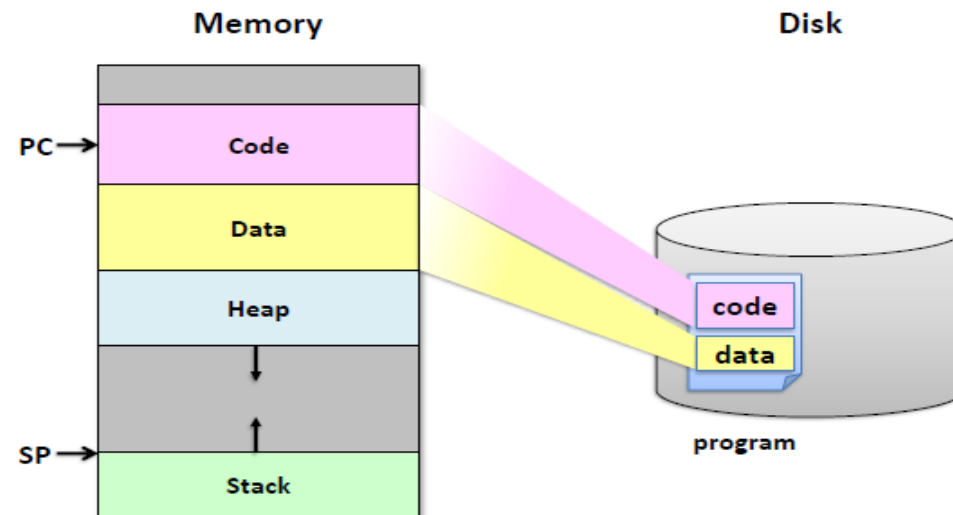
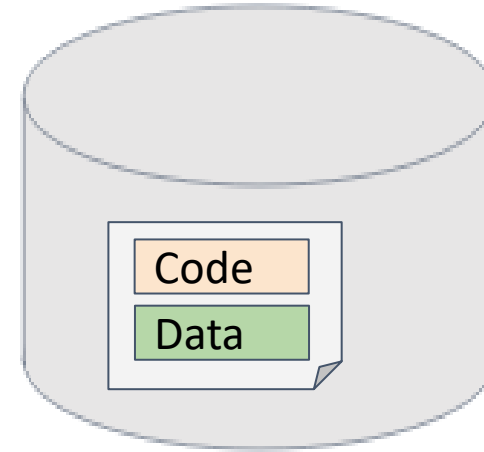
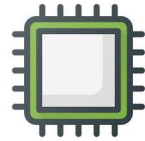
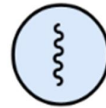
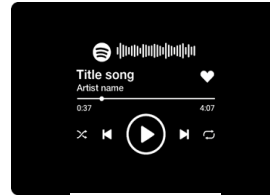
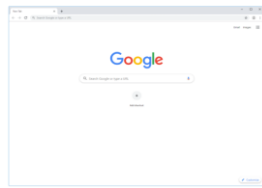
Programa



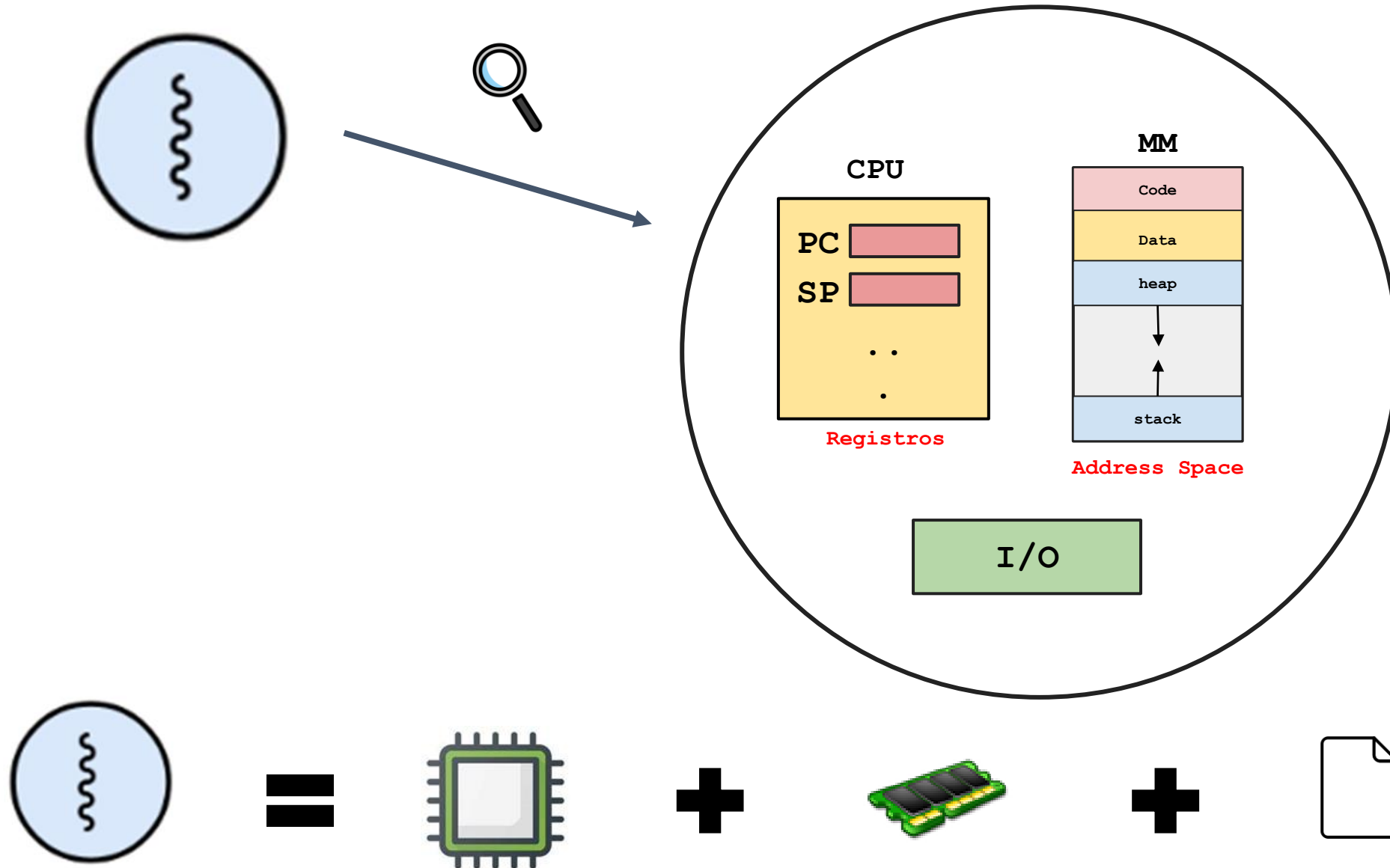
Proceso



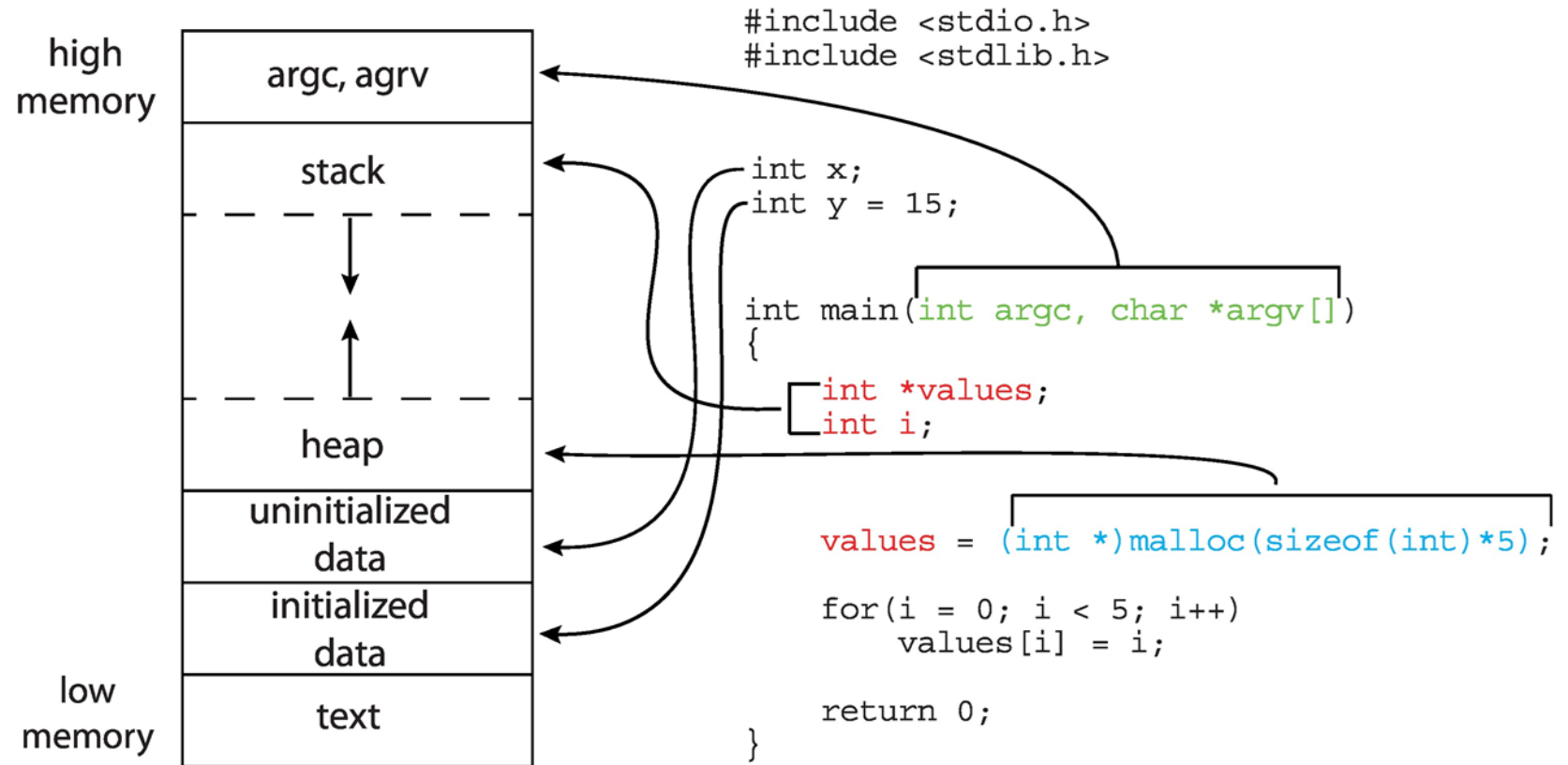
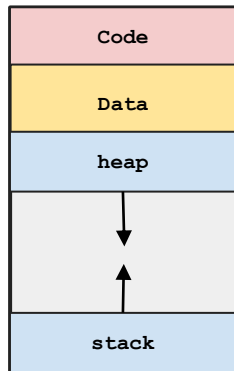
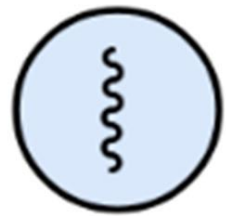
Programa. vs. proceso



Proceso: Abstracción



Virtualización

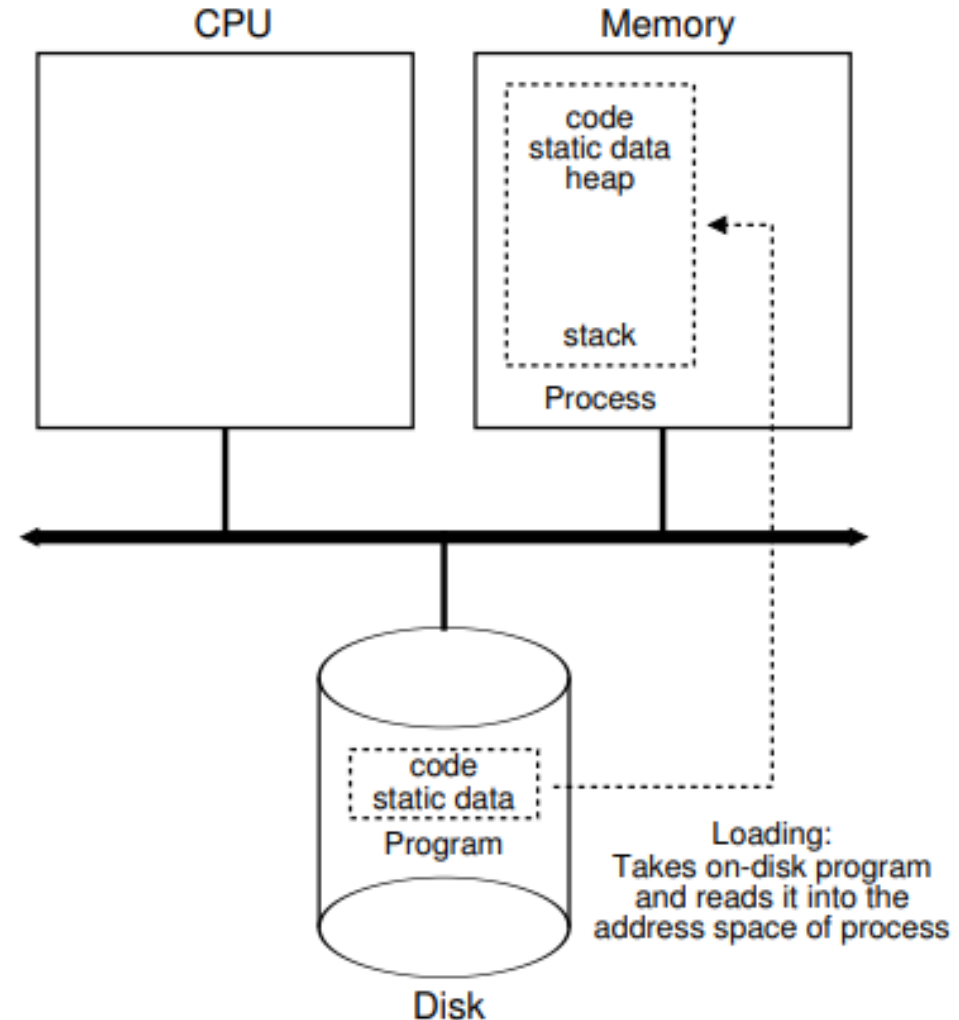


Simulación online ([link](#))

Carga: De un programa a un proceso

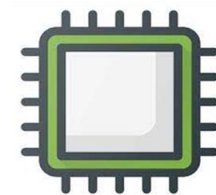
Pasos

1. del código del programa a memoria en forma diferida
2. Asignación de memoria en la pila (**stack**) en tiempo de ejecución
3. Creación del espacio de memoria **heap**
4. Realización de tareas de inicialización I/O
5. Inicio del programa ejecutando el punto de entrada `main()`

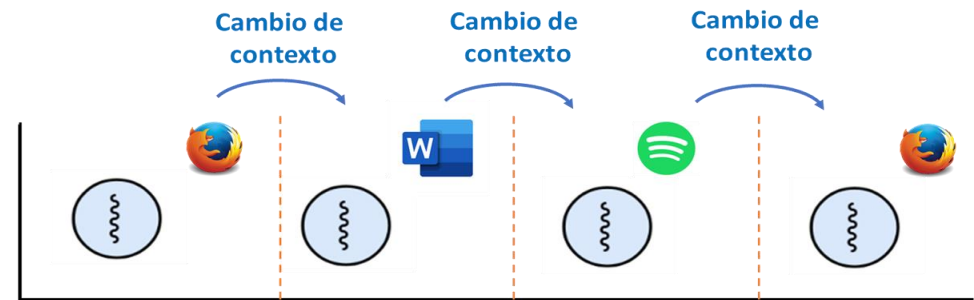


Volvamos a la pregunta

¿Cómo es que teniendo una sola CPU (en teoría) puedo correr varias tareas a la vez?



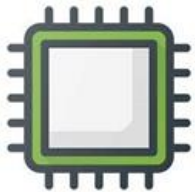
Técnica de **time sharing**



Elementos de los sistemas operativos

Estos tres conceptos son los pilares que permiten que el software interactúe con el hardware sin caos:

1. **Abstracción (lo que “ve” el usuario):** Interfaz simplificada que oculta la complejidad física del hardware.
 - process, thread, file, socket, memory page.
2. **Mecanismos (el “como” se hace):** representa las herramientas o funciones de bajo nivel que permiten implementar una operación.
 - create, schedule, open, write, locate.
3. **Políticas (El “quien” y “cuando”):** es el conjunto de reglas o algoritmos que deciden cómo se utilizan los mecanismos para cumplir un objetivo.
 - Last Recently Used (LRU), Earliest Deadline First (EDF), etc.

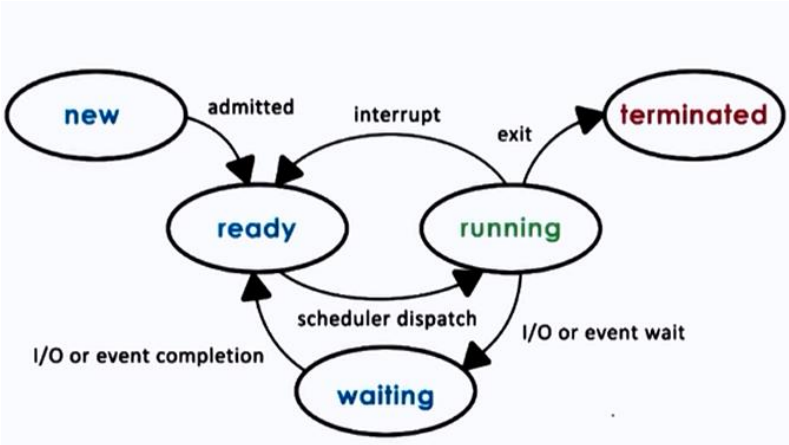


Process management	Abstracción	Proceso
	Mecanismos	create, destroy, wait, status,...
	Políticas	FIFO, RR,...

Como implementar la Abstracción

La Abstracción (El "Qué")

- **Definición:** Modelo simplificado que oculta la complejidad del hardware.
- **Objetivo:** Transformar recursos físicos (voltajes, sectores) en objetos lógicos.
- **Ejemplos:** Archivos, Procesos, Espacios de Direccionamiento.



La API (El "Contrato")

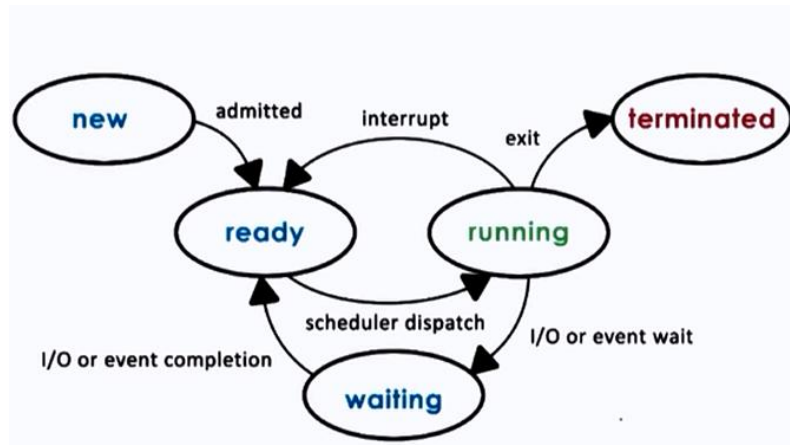
- **Definición:** Conjunto de funciones y protocolos (**punto de entrada**) para interactuar con el Kernel.
- **Rol:** Es la **formalización de la abstracción**. Permite la portabilidad (ej. POSIX).
- **Seguridad:** Única vía legal para pasar de *Modo Usuario* a *Modo Privilegiado*.

Función	Descripción
Crear (create)	Crea un nuevo proceso para ejecutar un programa.
Eliminar (destroy)	Forzar la detención de un programa en ejecución.
Esperar (wait)	Espera que un proceso termine su ejecución.
Operaciones de control	Ej. Suspender un proceso.
Estado (status)	Información de estado del proceso.

Como implementar la Abstracción

La Abstracción (El "Qué")

- **Definición:** Modelo simplificado que oculta la complejidad del hardware.
- **Objetivo:** Transformar recursos físicos (voltajes, sectores) en objetos lógicos.
- **Ejemplos:** Archivos, Procesos, Espacios de Direccionamiento.

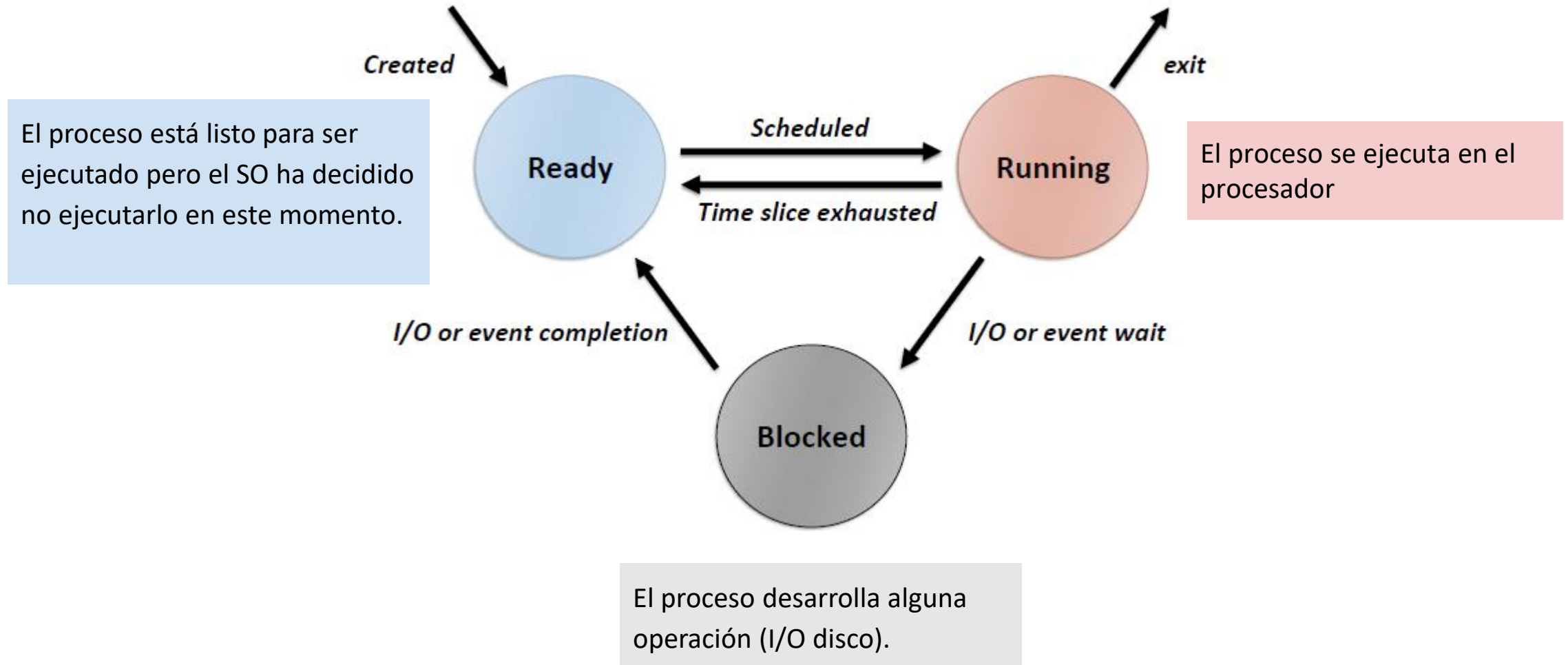


La API (El "Contrato")

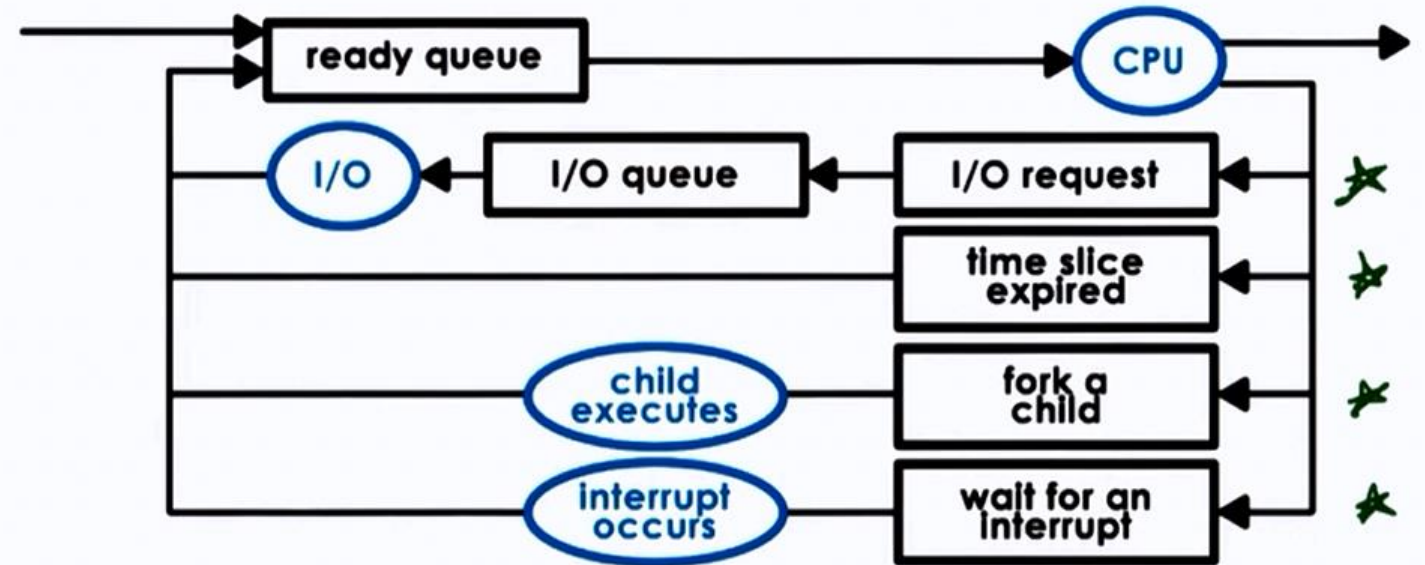
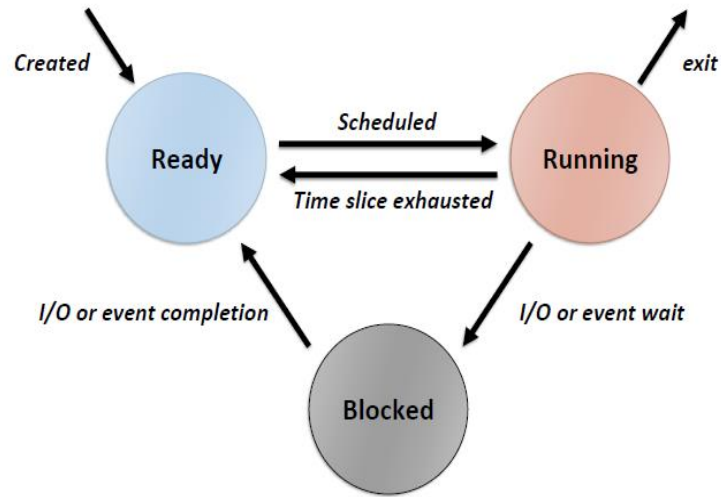
- **Definición:** Conjunto de funciones y protocolos (**punto de entrada**) para interactuar con el Kernel.
- **Rol:** Es la **formalización de la abstracción**. Permite la portabilidad (ej. POSIX).
- **Seguridad:** Única vía legal para pasar de *Modo Usuario* a *Modo Privilegiado*.

Función	Descripción
Crear (create)	Crea un nuevo proceso para ejecutar un programa.
Eliminar (destroy)	Forzar la detención de un programa en ejecución.
Esperar (wait)	Espera que un proceso termine su ejecución.
Operaciones de control	Ej. Suspender un proceso.
Estado (status)	Información de estado del proceso.

Modelo de tres estados



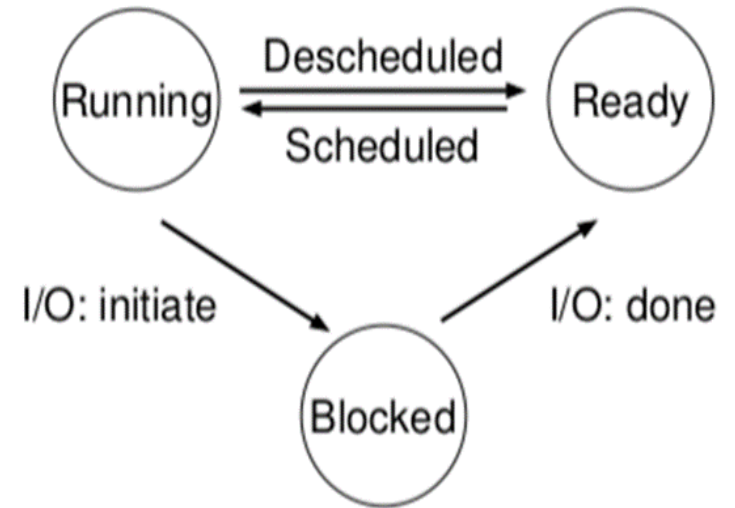
Modelo de tres estados



Estados de un proceso – Ejemplo 1

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	Process ₀ now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	Process ₁ now done

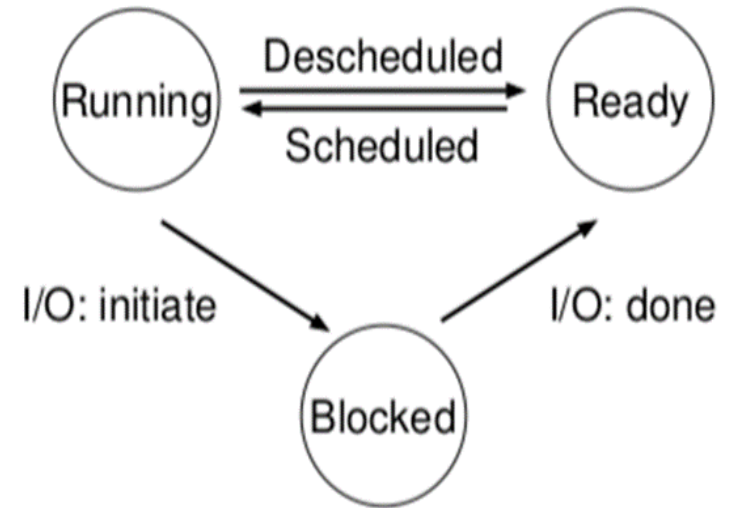
Tracing Process State: CPU Only



Estados de un proceso – Ejemplo 2

Time	Process ₀	Process ₁	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process ₀ initiates I/O
4	Blocked	Running	Process ₀ is blocked,
5	Blocked	Running	so Process ₁ runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process ₁ now done
9	Running	–	
10	Running	–	Process ₀ now done

Tracing Process State: CPU and I/O



Modelo Linux

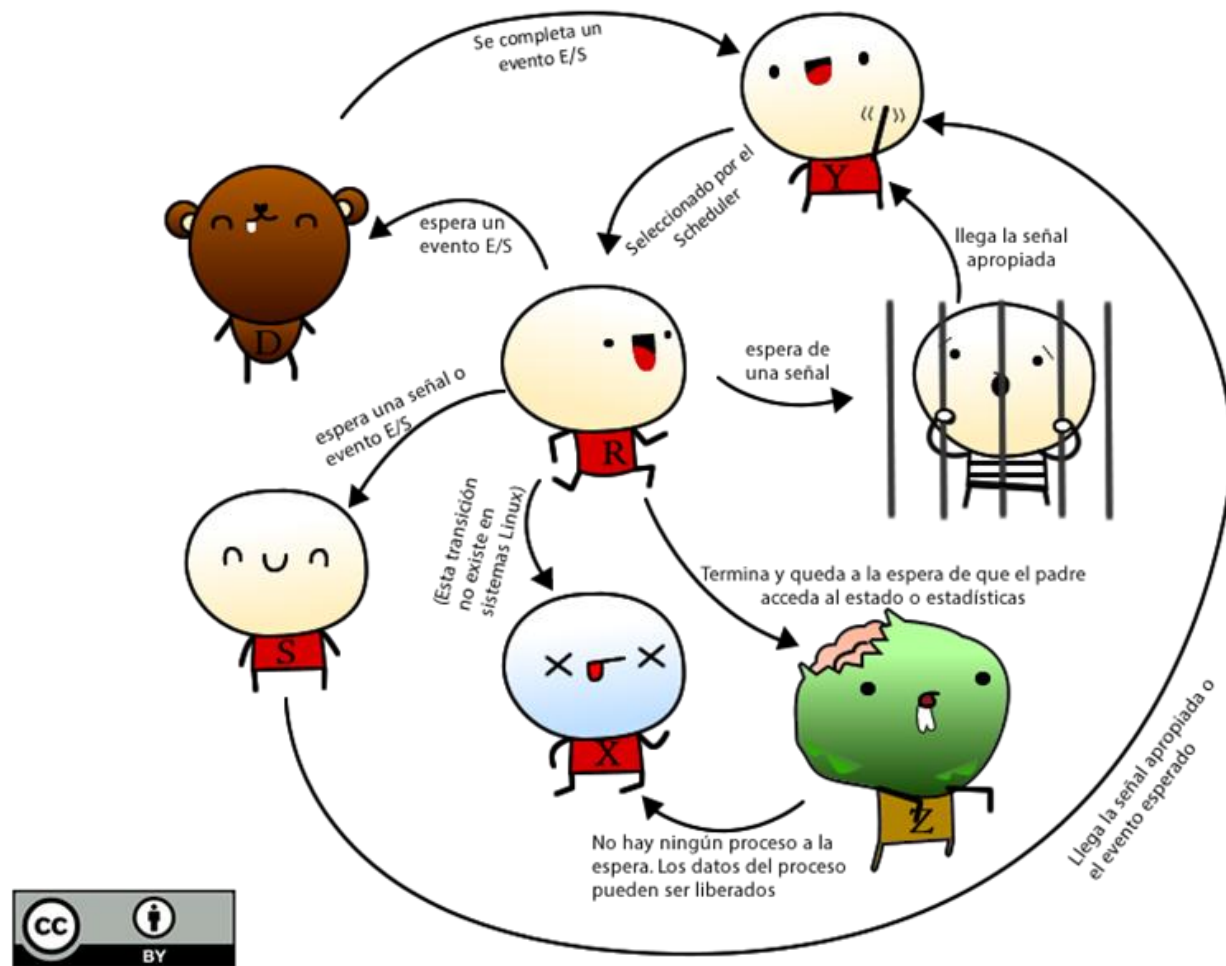
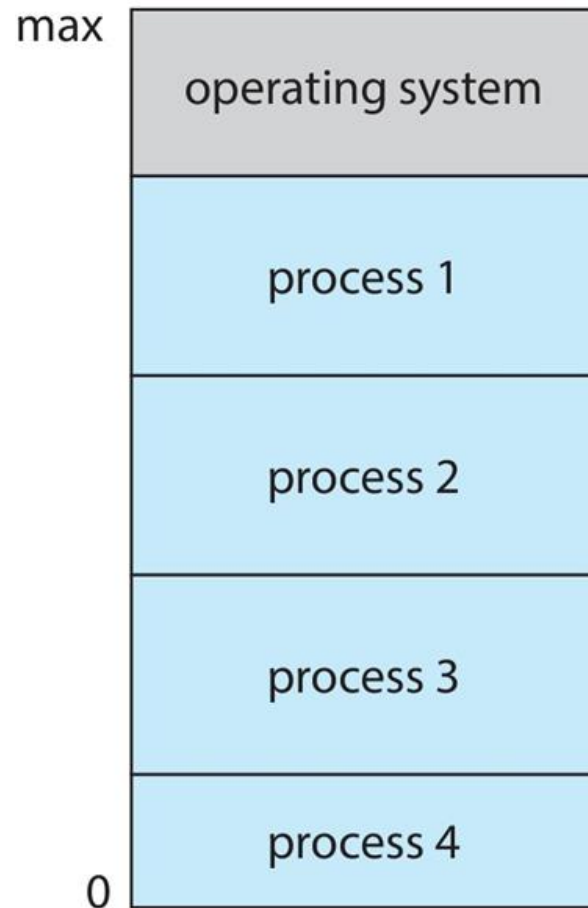


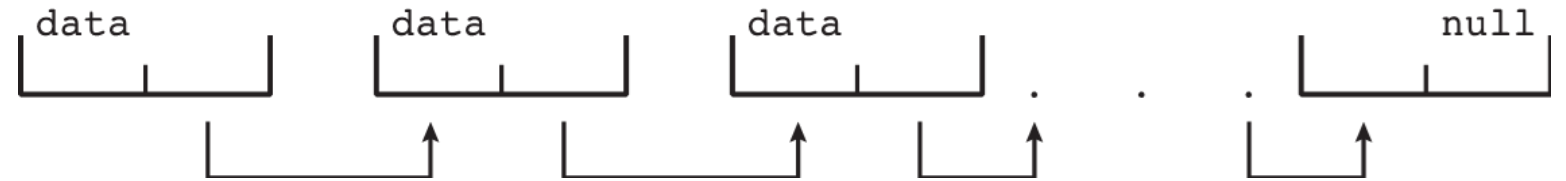
Imagen realizada por Adrián Martínez

Pasando al código – Estructuras de datos

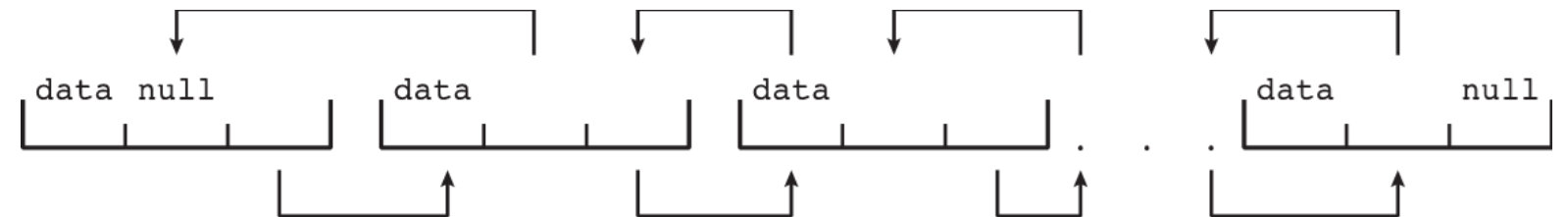
- El sistema operativo requiere **estructuras de datos** para rastrear los elementos claves.



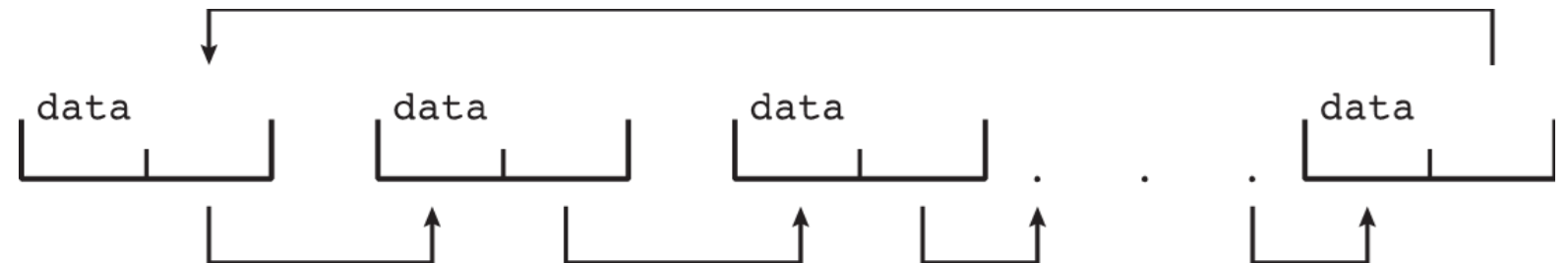
Single linked list



Double linked list

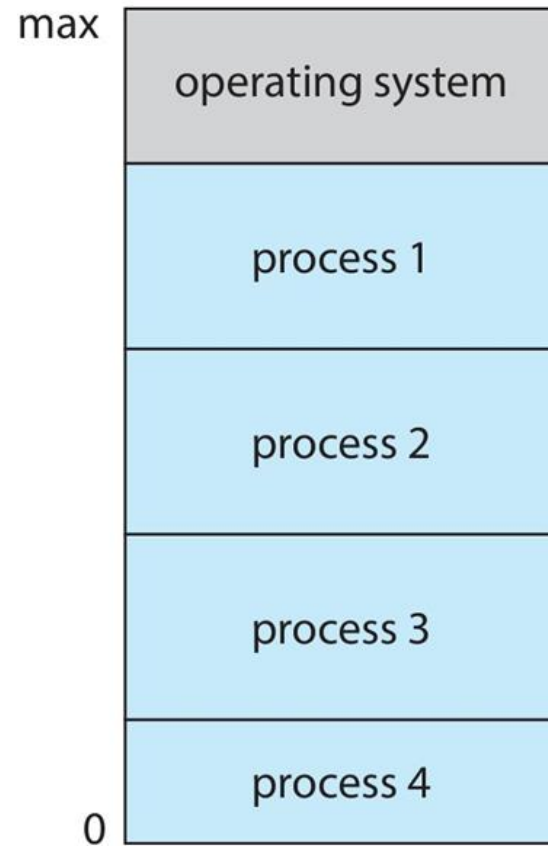


Circular linked list

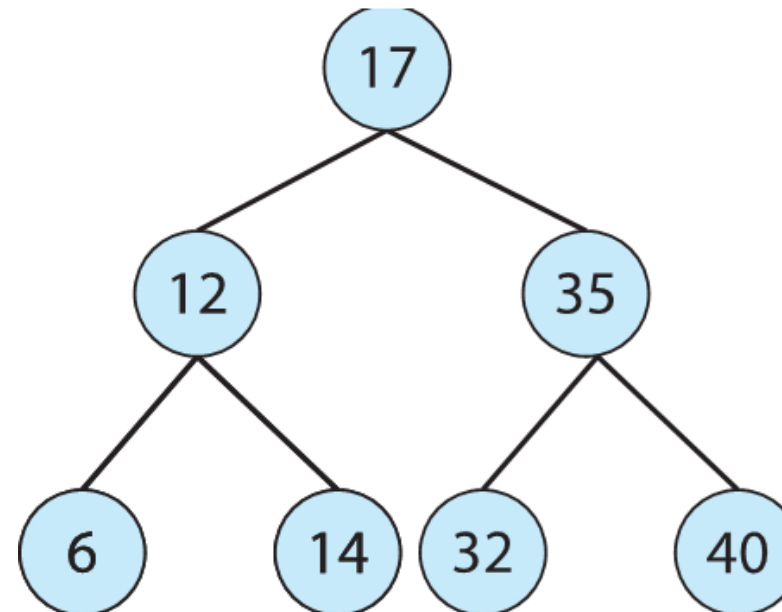


Pasando al código – Estructuras de datos

- El sistema operativo requiere **estructuras de datos** para rastrear los elementos claves.

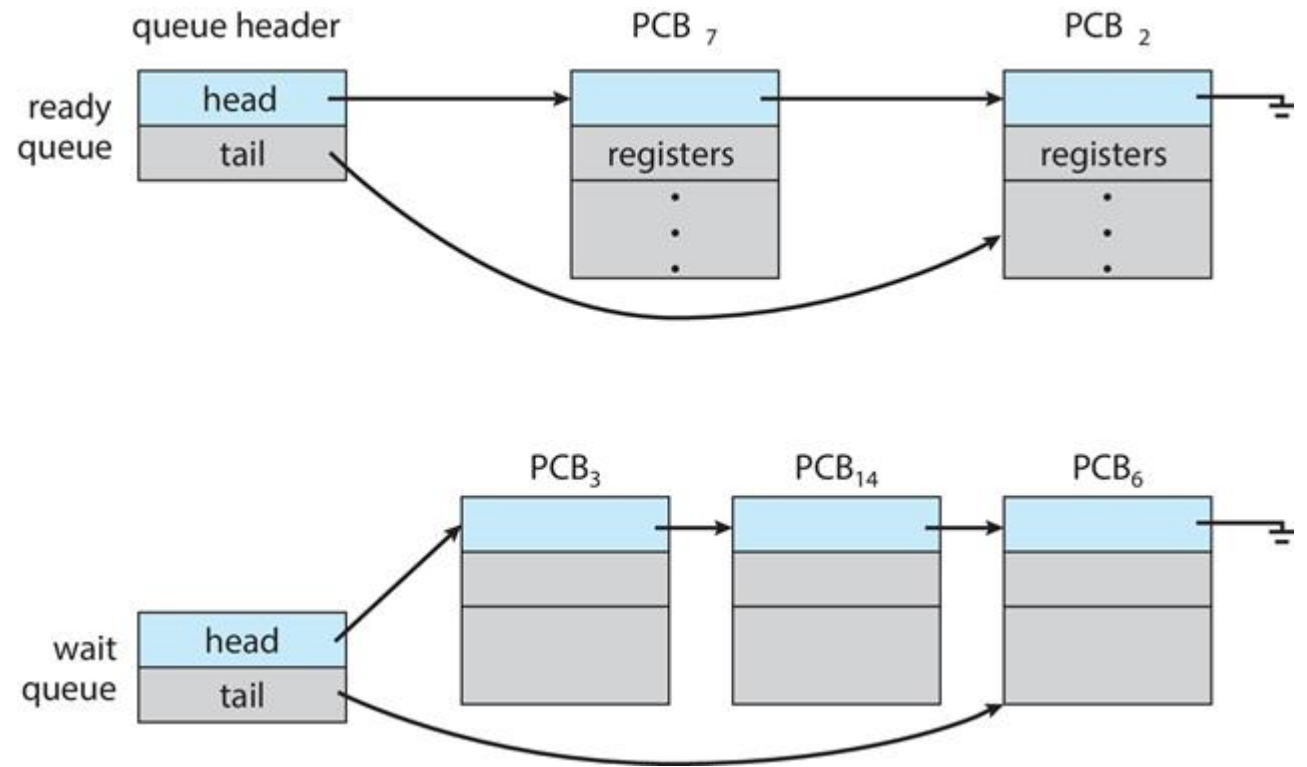
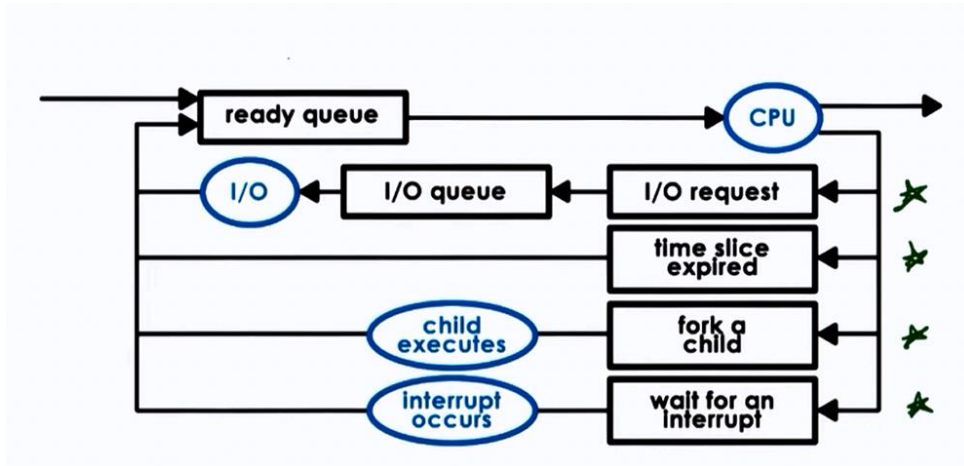


Binary search tree



Lista de procesos

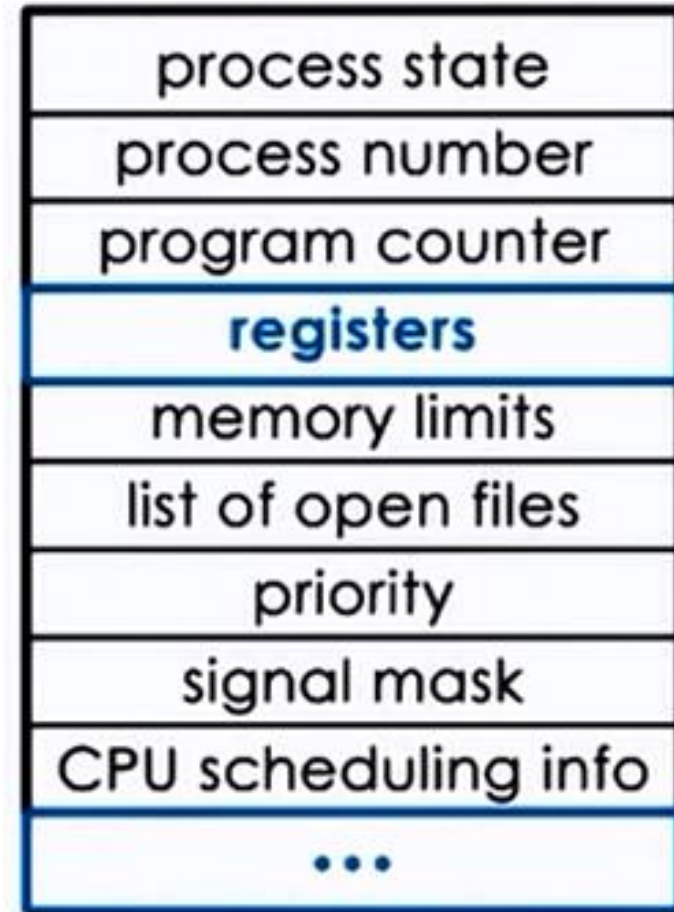
- Procesos listos
- Procesos bloqueados
- Procesos en ejecución



Process Control Block (PCB)

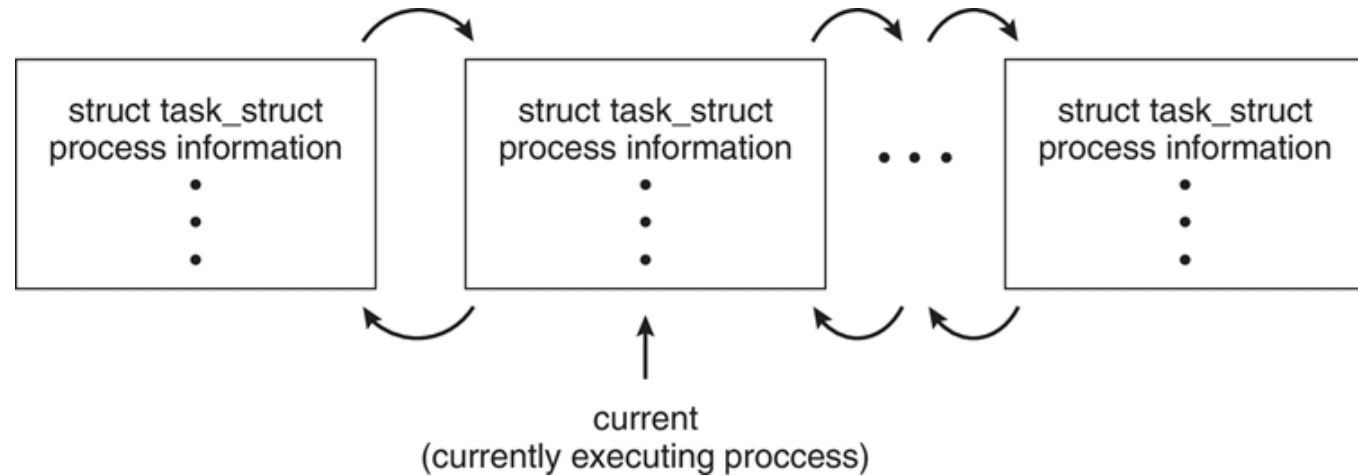
Información acerca de cada proceso (estructura en C).

- **Estado de ejecución del programa:** running, ready, etc.
- **Program counter:** Localización de la próxima instrucción a ejecutar.
- **Registros de la CPU:** Contenido de todos registros asociados al proceso.
- **Información de scheduling de la CPU:** Prioridades, etc.
- **Información de manejo de memoria:** Memoria asignada por el proceso.
- **Información (Accounting information):** CPU usada, tiempo gastado desde el inicio, limites de tiempo.
- **Información de estado de I/O:** Dispositivos I/O asociados al proceso, lista de archivos abiertos.



Process Control Block (PCB)

process state
process number
program counter
registers
memory limits
list of open files
priority
signal mask
CPU scheduling info
...

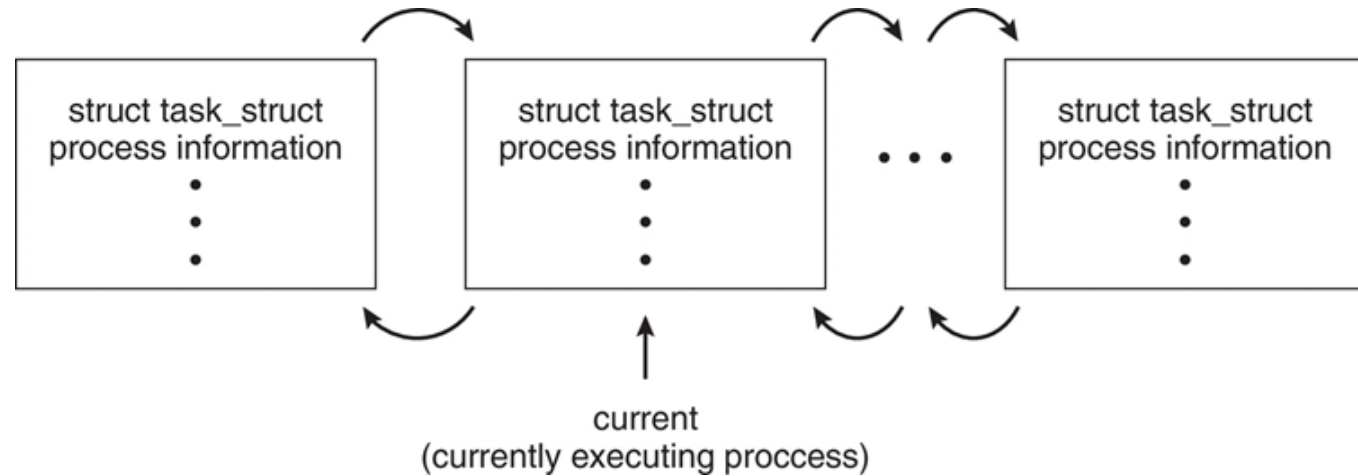


```
pid t_pid;    /* process identifier */
long state;   /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm;    /* address space of this process */
```

The xv6 Proc Structure ([link](#))

Process Control Block (PCB)

process state
process number
program counter
registers
memory limits
list of open files
priority
signal mask
CPU scheduling info
...



```
pid t_pid;    /* process identifier */
long state;   /* state of the process */
unsigned int time_slice /* scheduling information */
struct task_struct *parent; /* this process's parent */
struct list_head children; /* this process's children */
struct files_struct *files; /* list of open files */
struct mm_struct *mm;    /* address space of this process */
```

The xv6 Proc Structure ([link](#))

Process Control Block (PCB)

```
// the registers xv6 will save and restore
// to stop and subsequently restart a process
struct context {
    int eip;
    int esp;
    int ebx;
    int ecx;
    int edx;
    int esi;
    int edi;
    int ebp;
};

// the different states a process can be in
enum proc_state { UNUSED, EMBRYO, SLEEPING,
                  RUNNABLE, RUNNING, ZOMBIE };
```

The xv6 Proc Structure ([link](#))

Process Control Block (PCB)

```
// the information xv6 tracks about each process
// including its register context and state
struct proc {
    char *mem;                // Start of process memory
    uint sz;                  // Size of process memory
    char *kstack;             // Bottom of kernel stack
                                // for this process
    enum proc_state state;    // Process state
    int pid;                  // Process ID
    struct proc *parent;      // Parent process
    void *chan;               // If !zero, sleeping on chan
    int killed;               // If !zero, has been killed
    struct file *ofile[NOFILE]; // Open files
    struct inode *cwd;        // Current directory
    struct context context;    // Switch here to run process
    struct trapframe *tf;     // Trap frame for the
                                // current interrupt
};
```

The xv6 Proc Structure ([link](#))